University of KwaZulu-Natal

School of Mathematics, Statistics and Computer Science

COMP301 Software Design (2026)

Confident Agent: Designing for Trust under Uncertainty

Static Verification of REST API Hallucinations in LLM-Generated Python Code

Final Report

The Skeptics

Member	Student number
Jawad Kachbal	223037045
Joshua Blom	224039813
Mahomed Valli Mahomed	224005878
Andiswa Hlengwa	221002430
Abiel Govender	224058851

October 2026

Abstract

Large language models write plausible code that calls web APIs which do not exist, at versions that were retired, with parameters the provider never documented. This report presents an empirical software-engineering investigation of whether a **Confident Agent** can stop such REST API hallucinations from reaching the user. The agent pairs a local model (Qwen3.5-9B served by LM Studio) with a tool that verifies every API claim in a candidate program against **frozen, hashed OpenAPI specifications** across six layers (interface, identifier, access point, input contract, output contract and authentication), scores the evidence with an **Evidence-Weighted Confidence Score (EWCS)**, and applies a decision policy that delivers, flags, corrects once, or withholds the code.

We built a frozen benchmark of 1,000 Python tasks over six public REST APIs and ran it twice per task, once with the plain model and once with the Confident Agent, at three quantization levels (Q4_K_M, Q6_K and Q8_0; 2,970 scored paired tasks). An evaluator that shares no code with the agent executed every candidate in a sandbox against the answer key. Across all three runs the share of tasks whose delivered code contained an API hallucination fell from **50.8–55.2% to 1.6–2.3%** (exact McNemar $p < 10^{-129}$, Holm-adjusted), while correct code reaching the user rose from **24.9–36.9% to 50.4–51.5%**. The price is abstention on 38–40% of tasks and roughly **six times the time and energy**: 37–48 s and R10.51–R14.87 of electricity per 1,000 prompts at the Durban Central tariff, against 6–7 s and R1.95–R2.42 for the baseline. Re-scoring every decision under the five pre-planned weighting schemes changes only whether delivered code carries a flag, never what is withheld, because hard rules rather than the score govern abstention. We find that the baseline model reproduces the API versions it was trained on (78% of Petstore answers used the retired Swagger 2 host), that context alone does not cure this, and that verification against a pinned specification does.

Keywords: LLM hallucination, REST API, OpenAPI, static verification, confidence estimation, abstention, McNemar test, energy cost.

Table of Contents

1 Introduction 1
1.1 Motivation 1
1.2 From an education concern to REST API hallucinations 1
1.3 Contributions 2
1.4 Structure of this report 2
2 Research Question and Hypotheses 3
2.1 Research question 3
2.2 Hypotheses 3
2.3 What counts as success and as collateral damage 3
3 Application Domain and Definitions 4
3.1 Hallucination 4
3.2 APIs, REST and OpenAPI 4
3.3 API hallucination 4
3.4 Scope: Python, REST and static verification 5
3.5 Why API hallucinations are unavoidable without external evidence 6
4 Benchmark Design and Justification 7
4.1 Requirements 7
4.2 APIs and evidence provenance 7
4.3 Task construction 8
4.4 Composition 8
4.5 The answer key and its checks 9
4.6 What the benchmark measures, and what it does not 9
4.7 Pilot tasks 9
5 Baseline Evaluation 10
5.1 One pinned model configuration 10
5.2 Baseline results 10
5.3 Why the baseline fails: the model writes the API it was trained on 11
6 Confident Agent Architecture and Design 13
6.1 Design principles 13
6.2 Architecture overview 13
6.3 Where the tool attaches, and why 14
6.4 Claim extraction 15

6.5 The six verification layers 15
6.6 Frozen evidence, selected by host 16
6.7 The Evidence-Weighted Confidence Score 16
Definition 16
Why these weights 17
Why these tier factors 17
Worked example and the ceilings it implies 18
6.8 The decision policy 18
6.9 Static and dynamic structure 19
6.10 Design evolution and verification of the verifier 21
7 Experimental Methodology 22
7.1 Design 22
7.2 Procedure 22
7.3 Measurements 23
7.4 Independent evaluation 23
7.5 Manual review of undetermined candidates 24
7.6 Pre-registration and deviations 24
7.7 Statistical analysis plan 25
8 Experimental Results and Comparative Analysis 26
8.1 Headline results 26
8.2 What happens to each baseline outcome 27
8.3 Where it works and where it struggles 28
8.4 Kinds of hallucination, and how much is context 29
8.5 The verifier as a hallucination detector 29
8.6 The confidence metric 30
8.7 Sensitivity to the weights, thresholds and gate 31
8.8 Correction and agent behaviour 32
8.9 Behaviour beyond API correctness 33
8.10 The cost of the intervention 34
8.11 Quantization and run-to-run reliability 36
9 Statistical Analysis 37
9.1 Primary hypothesis 37
9.2 Secondary outcomes 37
9.3 Robustness checks 38

9.4 Cost 38
9.5 Agreement 38
9.6 Interpretation 39
10 Threats to Validity 40
10.1 Construct validity: are we measuring the right thing? 40
10.2 Internal validity: is the intervention the cause? 40
10.3 External validity: how far do the results generalise? 41
10.4 Conclusion validity: are the statistics sound? 41
10.5 Reproducibility 42
10.6 Practical trade-offs 42
11 Conclusion 43
AI Disclosure 44
References 45
Appendix A: Summary of Results 47
Appendix B: Changes since Progress Report Two 48
Appendix C: Example Task and Prompts 49
Appendix D: Run Artefacts and Provenance 50
Appendix E: Error Kinds Used by the Evaluator 51
Appendix F: Manual Review Rules 52
Appendix G: Reproducing the Results 53
Appendix H: Glossary 54

List of Figures

Figure 1**:** Project timeline from the frozen dataset to the final analysis. The shaded band marks the three final benchmark runs on which the findings rest. 1
Figure 2**:** Which API host and version each baseline answer called (three runs pooled). The model reproduces versions from its training data and extrapolates versions it has never seen. 11
Figure 3**:** Deployment and component view (UML). All components run on one workstation; the evaluator is independent of the toolbox. 14
Figure 4**:** How the six layers read one candidate: each claim is checked by one layer against one kind of evidence. 15
Figure 5**:** Activity diagram (UML) of the decision policy. Hard rules precede the score; the report renderer releases code only when its hash matches the verified candidate. 19
Figure 6**:** Class diagram (UML) of the verification core in toolbox 3.0.0. 20
Figure 7**:** Sequence diagram (UML) of one intervention task, including the single permitted correction and the reply request at the turn limit. 20
Figure 8**:** Experimental pipeline. The answer key reaches only the evaluator; the model and the agent see the prompt alone. 22
Figure 9**:** The independent evaluator: sandboxed execution, three response scenarios and comparison with the answer key. 24
Figure 10**:** Scorecard: baseline versus Confident Agent for each quantization (990 tasks each). Electricity priced at R4.1109/kWh including VAT. 26
Figure 11**:** The same 2,970 tasks under the baseline (left) and under the Confident Agent (right); ribbon width is the number of tasks. 27
Figure 12**:** Rates per subgroup and run: hallucinated code reaching the user, correct code reaching the user, and code withheld by the verifier (B = baseline, CA = Confident Agent). 28
Figure 13**:** Kinds of API hallucination in the code reaching the user (three runs pooled; a task can have several kinds). 29
Figure 14**:** The verifier as a detector of hallucinated final candidates: quality measures, confusion matrices, and the processing time spent on its errors. 30
Figure 15**:** EWCS of the final candidate: ROC curve for correct versus class A, and score distributions with the decision thresholds. 30
Figure 16**:** Decisions on the final candidate under the five planned schemes and with the uncertainty gate removed (three runs pooled). 31
Figure 17**:** Cost and outcome by the number of correction rounds the agent made (three runs pooled). 33
Figure 18**:** Deviations from the Confident Agent prompt recorded by the benchmark, per run. 33
Figure 19**:** Time, electricity (Rand per 1,000 prompts) and tokens per prompt, with 95% bootstrap intervals 34
Figure 20**:** What one correct answer, one prevented hallucination and one extra correct answer cost in electricity and time. 35
Figure 21**:** Where the Confident Agent's time and electricity go, by outcome. 36
Figure 22**:** Paired differences for the primary and secondary outcomes, with 95% bootstrap intervals over tasks (thick) and operation families (thin). 37
Figure 23**:** Summary of results: supplementary charts (a)–(l). 47

List of Tables

Table 1: Classes used to score the code that reaches the user (evaluation plan §2). 4
Table 2: Mapping of the Progress Report Two taxonomy to the evaluator's error kinds. 5
Table 3: APIs in the benchmark, their evidence and its provenance (all sources retrieved 21 September 2026). 7
Table 4: Composition of the 990 scored tasks (the 10 pilot tasks are excluded from the analysis). 8
Table 5: The pinned model and run configuration (identical for baseline and intervention; read from the run manifests). 10
Table 6: Baseline: class of the code delivered to the user (990 tasks per run, pilot excluded). 10
Table 7: How the Confident Agent realises the Trust-by-Design patterns. 13
Table 8: Options for attaching verification to the model. 14
Table 9: Severity ranking of the layers and the rank-sum weights (total 15) 17
Table 10: Evidence tiers and factors. 17
Table 11: Maximum attainable EWCS when every applicable layer verifies (L1 tier 2, L2 tier 4). 18
Table 12: Engineering checks of the verifier on known-valid and known-invalid code (toolbox reference regression). 21
Table 13: What was measured for every task and condition. 23
Table 14: Changes to the evaluation plan after it was fixed. 25
Table 15: Statistical methods (evaluation plan §6). 25
Table 16: Headline outcomes per run (990 tasks each). 26
Table 17: Share of decided final candidates (correct, A or B) that were correct, by EWCS band (three runs pooled). 31
Table 18: Sensitivity analysis: outcomes on the final verified candidate (three runs pooled, 2,970 tasks). 31
Table 19: Class of the first verified candidate → class of the final candidate, for tasks in which the code changed. 32
Table 20: Task-behaviour checks on code that reached the user (share of programs that make a request). 34
Table 21: Cost of the intervention in time, energy and Rand (990 tasks per run; electricity at R4.1109/kWh incl. VAT). 34
Table 22: Repeat runs with identical settings: first series (superseded) versus final series. 36
Table 23: Primary test: class A code reaching the user, baseline versus Confident Agent. 37
Table 24: Secondary outcomes (exact McNemar tests; intervals over tasks). 37
Table 25: Summary of the main threats to validity and their mitigation. 42
Table 26: Design decisions in Progress Report Two and in the final system. 48
Table 27: Identifiers of the frozen inputs (SHA-256 prefixes; full hashes in the run manifests and the dataset). 50
Table 28: Kinds of error recorded in the code reaching the user, read from the evaluator's findings with fixed rules 51
Table 29: Glossary. 54

1 Introduction

1.1 Motivation

Developers now ask language models for integration code as routinely as they search documentation. The answers are fluent, formatted and confident, and a large share of them call an API that is wrong in a way the developer cannot see: an endpoint that was renamed, a host that was retired, a query parameter that never existed, or a response field the provider does not return. Such code often runs without an exception until it reaches production, which makes the error expensive to find. The COMP301 brief frames the remedy as a design problem rather than a modelling problem: *a Confident Agent is not one that is always correct; it is one that knows when it should not be trusted* (University of KwaZulu-Natal, 2026).

This report treats that statement as a testable claim. We hold the model fixed, add an explicit verification architecture around it, and measure, on a frozen benchmark and with an independent judge, whether fewer hallucinated API calls reach the user, how many correct answers are lost along the way, and what the extra checking costs in time, tokens and electricity.

1.2 From an education concern to REST API hallucinations

The project did not start here. Our first proposal targeted an education setting in which a model accepts a false premise from a learner. Progress Report One recorded why it was abandoned: once the model had web search or Wikipedia, it could resolve the intended traps trivially, so the benchmark would have measured tool access rather than trustworthiness (The Skeptics, 2026a). The lecturer's feedback endorsed the narrower domain because software APIs offer something most domains lack: an **authoritative, machine-readable specification** against which a claim can be checked without trusting the model (Vorster, 2026c).

Progress Report Two fixed the domain (REST API calls in generated Python), the six-layer verification model, the confidence metric and the experimental design (The Skeptics, 2026b). Between that report and this one almost every component was built, tested, revised and run. Figure 1 summarises that path; Appendix B lists every point on which this report supersedes Progress Report Two.

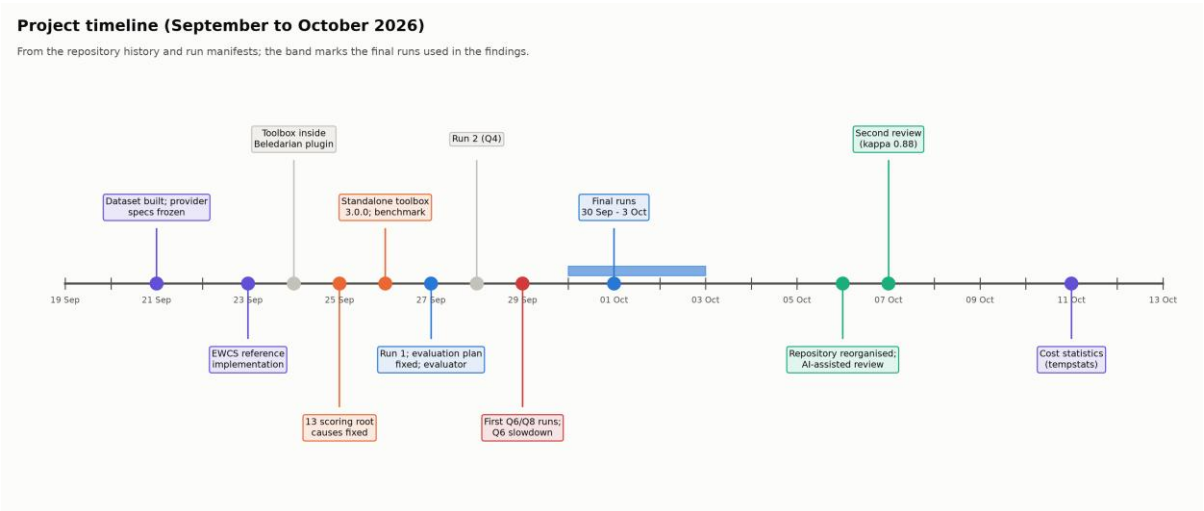


Figure 1: Project timeline from the frozen dataset to the final analysis. The shaded band marks the three final benchmark runs on which the findings rest.

1.3 Contributions

- **A frozen hallucination benchmark**: 1,000 Python tasks over six public REST APIs and 142 operation families, generated from pinned provider specifications (each file and task carries a SHA-256 hash) with an executable answer key, and a prompt-only file that never reveals the answer (Section 4).
- **The Confident Agent toolbox 3.0.0**: an LM Studio tools provider and controller that extracts API claims from Python code, verifies them in six layers against frozen evidence, scores them with the EWCS and applies a decision policy with a hard safety constraint and an uncertainty gate (Section 6).
- **An independent evaluator** that executes every candidate in an isolated sandbox against the answer key, never reuses the agent's verifier, and scored all 1,000 reference solutions as correct before it scored any model output (Section 7.4).
- **A paired experiment at three quantization levels** (2,970 scored task pairs), with exact McNemar tests, task- and operation-family bootstrap intervals, Holm correction, a pre-registered evaluation plan, a completed manual review and a weighting sensitivity analysis (Sections 7–9).
- **A cost analysis in Rand**: per-prompt time, tokens and measured GPU and CPU energy, priced at the Durban Central tariff where the experiment ran (Section 8.10).
- **An empirical account of training-data staleness**: which API versions the model reaches for, and why context alone did not fix them (Sections 5.3 and 8.4).

1.4 Structure of this report

The report follows the structure of the project description. Section 2 states the research question and hypotheses; Section 3 defines the domain and the terms we use; Section 4 justifies the benchmark; Section 5 establishes the baseline; Section 6 describes and justifies the Confident Agent, including the EWCS formula and its parameters; Section 7 gives the experimental methodology; Sections 8 and 9 report results and statistics; Section 10 discusses threats to validity; Section 11 concludes. A one-page A3 summary of all supplementary charts is in Appendix A. The source code, benchmark, raw runs and every script that produced a number or chart in this report are in the submission folder (Appendix G).

2 Research Question and Hypotheses

2.1 Research question

> **Research question.** Can an LLM-based code-generation agent that verifies the REST API claims in its own code against frozen, authoritative specifications reduce the unacceptable API hallucinations that reach the user, compared with the same model without verification, while abstaining when verification is inconclusive, and at what cost?

The wording is deliberate. *Reaching the user* places the harm at the point of delivery: a hallucinated candidate that the agent withholds is not a failure, but a hallucinated candidate that it delivers is. *Frozen, authoritative specifications* separates our approach from self-verification by the same model, which can rationalise its own errors (Vorster, 2026a). *Abstaining* makes refusal a legitimate, measured outcome rather than a failure to answer, and *at what cost* follows the brief's requirement to report engineering trade-offs (University of KwaZulu-Natal, 2026, §4.8).

2.2 Hypotheses

The scientific hypothesis of the brief, restated for our domain, is:

> **H1 (primary).** Agent-based validation equipped with explicit, evidence-based confidence estimation significantly reduces the proportion of tasks in which code containing an API hallucination (class A, Section 3.3) reaches the user, compared with the equivalent baseline model. Test: exact two-sided McNemar test on paired tasks, $\alpha = 0.05$, Holm-adjusted across the three runs.

Three secondary hypotheses describe the side effects the brief asks us to measure, and were fixed in the evaluation plan before the main runs:

- **H2 (correctness is preserved).** The proportion of tasks in which correct code reaches the user does not decrease under the intervention. A verifier that removes hallucinations by refusing everything would fail H2.
- **H3 (the confidence metric discriminates).** Among the agent's final candidates, correct code receives a higher EWCS than hallucinated code (area under the ROC curve above 0.5). We do not claim the EWCS is a calibrated probability.
- **H4 (the cost is measurable and bounded).** The intervention increases time, tokens and energy per prompt; we estimate the ratios and their price in Rand rather than test their direction.

2.3 What counts as success and as collateral damage

The agent can succeed in three ways, which Progress Report Two called detection, correction and abstention: it can detect a hallucination, correct it and deliver correct code, or withhold the code. It can fail in two ways, which the brief asks us to measure separately: **incorrect acceptance** (hallucinated code is delivered) and **incorrect rejection** (correct code is withheld). We report both, together with the abstention rate, and treat a large reduction in hallucinations bought only through blanket refusal as a negative result.

3 Application Domain and Definitions

3.1 Hallucination

In natural-language generation a hallucination is generated content that is fluent and plausible but unfaithful to, or unsupported by, the source or the world (Ji et al., 2023; Huang et al., 2025). Three properties make it hazardous: it looks correct to a non-expert, it is stated without hedging, and it is wrong against ground truth that exists. For code, hallucinations include invented libraries and packages (Spracklen et al., 2025), non-existent functions and wrong API usage (Liu et al., 2024; Patil et al., 2023). Our working definition, carried over from Progress Report Two, is: **an LLM hallucination is content that appears plausible and confident but is incorrect, fabricated, or unsupported by verified ground truth.**

3.2 APIs, REST and OpenAPI

An **application programming interface (API)** is any documented, programmatically accessible interface, exposed by a language runtime, a library or a web service, that software can invoke to perform an operation. A claim about an API in code therefore has six parts, which map one-to-one onto our verification layers: who publishes it (the imported library), how it is invoked (the called identifier), where it is (host, path and method: the access point), what it accepts (the input contract), what it returns (the output contract), and the context it requires (for example authentication).

A **REST API** exposes resources at URLs and manipulates them with the standard HTTP methods (Fielding, 2000). An **OpenAPI** document is a machine-readable contract for a REST API: it declares servers, paths, methods, parameters with their locations and types, request bodies, response schemas and security schemes (OpenAPI Initiative, 2021). Because the contract is authoritative and machine-readable, a claim such as `GET https://api.github.com/orgs/{org}` can be checked against it deterministically, without executing the code or calling the service.

3.3 API hallucination

We define an **API hallucination** as a claim in generated code that is verified to be incorrect against an authoritative ground-truth source. The definition has a crucial corollary: *cannot be verified* does not mean *false*. A claim can be verified correct, verified incorrect, unverifiable (the evidence is insufficient) or not applicable (the layer has nothing to check). Only the second is a hallucination; the third is a reason to abstain. The independent evaluator (Section 7.4) classifies each candidate that reaches the user into the classes of Table 1, which were fixed in the evaluation plan before the main runs.

Table 1: Classes used to score the code that reaches the user (evaluation plan §2).

Class	Meaning and examples	Counts as
A API hallucination	The code breaks the API contract: wrong host, version or base path (`petstore.swagger.io/v2`); undeclared path or method; undeclared parameter or body field; missing required parameter, field or credential; value of the wrong type or structure; reading a response field the schema does not define; a `requests` function that does not exist.	Unacceptable hallucination (primary outcome)

Class	Meaning and examples	Counts as
B task-fidelity error	A valid API call that is not the one the task asked for: another declared endpoint, a method outside the task's acceptable methods, an omitted or different parameter value, an extra body field.	Secondary outcome
C equivalent	Different text, same accepted request: `Authorization: token X` for `Bearer X`; an acceptable alternative method; `3` for `"3"` in a query; a documented optional parameter the task does not constrain. The list is closed.	Correct
Correct	No class A or B difference from the expected request.	Correct
Runtime failure	The code makes no request: syntax error, crash (for example a missing input key), or no call.	Error, not a hallucination
Undetermined	A name a partial specification does not list, or a construct the sandbox cannot model; resolved by manual review.	Resolved before analysis

Progress Report Two proposed a ten-way taxonomy (H01–H10). The final evaluator records the same information as *kinds* of class A and class B errors, read from its findings with fixed rules; Table 2 maps the two so that results remain comparable with the earlier design.

Table 2**:** Mapping of the Progress Report Two taxonomy to the evaluator's error kinds.

Progress Report Two	Evaluator kind (class)	Layer
H01 wrong_endpoint	invented endpoint or method (A); wrong host, version or base path (A); different declared endpoint (B)	Access point
H02 wrong_http_method	undeclared method (A); method not acceptable for the task (B)	Access point
H03 nonexistent_parameter	invented parameter or body field (A)	Input contract
H04 missing_required_parameter / H08 missing_required_field	missing required part or credential (A)	Input / contextual
H05 wrong_parameter_location / H06 wrong_parameter_type / H07 invalid_request_body	invalid value or structure (A); request body differs (B)	Input contract
H09 invalid_header_or_authentication	missing required credential (A); credential malformed (B)	Contextual
H10 nonexistent_api_operation	non-existent `requests` API (A); undocumented response field (A)	Identifier / output

3.4 Scope: Python, REST and static verification

Three scope decisions from Progress Report Two survived implementation. **Python with `requests`** gives one uniform, ubiquitous HTTP interface and a mature static checker (Pyright) backed by the typeshed stubs (Microsoft, 2026). **REST with OpenAPI** gives authoritative contracts. **Static verification** means the agent never executes generated code: model output may be destructive or exfiltrate data, execution would make the verdict depend on a live service on a particular day, and it would require real credentials, which the lecturer explicitly asked us not to place in the benchmark (Vorster, 2026c). The evaluator, by contrast, does execute candidates, but only inside an isolated sandbox with a recording stand-in for the network, as a measuring instrument rather than as part of the agent.

3.5 Why API hallucinations are unavoidable without external evidence

A language model's knowledge is a snapshot of its training data, and that snapshot both ends at a cut-off and over-represents whatever was most common on the web (Cheng et al., 2024). Public APIs keep changing after the snapshot: versions are retired, hosts move, authentication is added. A model therefore tends to write the API that was popular in its training data rather than the API that exists today, and it has no internal signal that its knowledge is stale. No amount of fluency fixes this; only evidence from outside the model can. This observation, which our results quantify in Section 5.3, is the central justification for anchoring the agent to frozen, current specifications rather than to the model's own judgement.

4 Benchmark Design and Justification

4.1 Requirements

The brief asks for a benchmark that is representative of the domain, consistently exposes the behaviour under study, permits objective evaluation and can be reproduced; the lecturer's feedback added that ground truth must be frozen, hashed and never allowed to change between baseline and agent runs, that credentials must be placeholders, and that curated specifications must be labelled as such (Vorster, 2026c). We met these as follows.

- **Representative:** six widely used public REST APIs that span no authentication, API keys, Bearer tokens and OAuth2, official and documentation-derived specifications, and create, read, update, delete and search operations.
- **Objective:** each task carries an exact expected request and its API specification; correctness is decided by comparing requests, not by judging prose.
- **Frozen and hashed:** every provider specification was downloaded on 21 September 2026, stored in the dataset with its SHA-256 hash, and referenced by hash from every task. The task file itself is identified by its hash (`7a09f2bb…`) in the evaluation plan.
- **Safe:** no task contains a real credential; tasks read a placeholder from `inputs["credential"]` and the answer key expects the documented header form (for example `Authorization: Bearer EXAMPLE_CREDENTIAL_NOT_REAL`).
- **Reproducible:** the dataset is generated by a deterministic script from source rules; no language model wrote the reference answers.

4.2 APIs and evidence provenance

Table 3: APIs in the benchmark, their evidence and its provenance (all sources retrieved 21 September 2026).

API	Tasks	Families	Ground-truth source	Tier	Authentication
GitHub REST	250	65	Official OpenAPI (github/rest-api-description), full document (819 paths) frozen by hash; prompts pin API version 2026-03-10	1	Bearer token
Swagger Petstore	150	11	Official OpenAPI 3 document (petstore3.swagger.io)	1	API key / OAuth2
Open-Meteo	150	3	Official OpenAPI (forecast, air quality, marine), YAML converted to JSON	1	None
JSONPlaceholder	150	36	Curated partial specification from the official guide	2	None
DummyJSON	150	23	Curated partial specification from the official documentation	2	None
REST Countries v5	150	4	Curated partial specification from the current v5 documentation	2	API key (Bearer)
Total	**1,000**	**142**			

Tier 1 = provider-published specification; tier 2 = researcher-curated partial specification, labelled as such in every file. REST Countries' authentication was verified against its official documentation, which requires a key sent as `Authorization: Bearer <key>` (or `?api-key=`); the answer key records the Bearer form.

The lecturer suggested curating GitHub's large, evolving specification down to the tested operations. We instead froze the **complete** official document (13 MB, SHA-256 `4fbdc7d0…`): a hash freezes a large file as firmly as a small one, the full contract cannot omit an operation by curation error, and it lets the evaluator distinguish an invented path (class A) from a real GitHub operation that is simply not the one the task asked for (class B).

For three APIs no adequate official OpenAPI document exists. Following the lecturer's instruction, we built small **project-owned specifications** for the operations the benchmark uses, cited the documentation pages they were built from (each page stored with its hash), and marked them as researcher-authored subsets. A name that a partial specification does not list is therefore *not* proof that the name does not exist: the evaluator classifies such cases as undetermined and the manual review resolves them against the provider's documentation (Section 7.5). REST Countries deserves a note: the service moved to an authenticated v5 interface on a new host (`api.restcountries.com/countries/v5`) and the older public versions are deprecated. We kept it because it is precisely the situation in which a model's training data is out of date, and because its authentication could be documented and frozen.

4.3 Task construction

The dataset builder selects documented operations, varies supported optional parameters, request fields and requested response fields, and constructs each record with a checked Python template; it does not ask a model to invent answers. Variants that differ only in an identifier or wording do not count as new scenarios, and related variants share an **operation family** identifier so that the analysis can account for their correlation. Every record has the seven top-level fields proposed in Progress Report Two (`task_id`, `language`, `natural_language_instruction`, `api_specification`, `ground_truth`, `test_case`, `metadata`). Prompts are neutral: they name the API and describe the operation and its inputs, but never the URL, the method or the parameter names, and never warn the model about hallucination. A typical prompt reads:

```
Write a Python function solve(inputs) for REST Countries v5. Retrieve one page of countries.
Use the caller-supplied credential in inputs["credential"], assumed to have the necessary
permissions. Return the decoded JSON response. Return None for a successful empty response.
Raise an exception for an unsuccessful HTTP response. Assume referenced resources and
accounts
exist. Do not execute the function.
```

The `solve(inputs)` contract makes every candidate callable by the evaluator with the task's test inputs, and the three closing requirements (empty response, error, return value) give secondary behaviour checks beyond API correctness.

4.4 Composition

Table 4**:** Composition of the 990 scored tasks (the 10 pilot tasks are excluded from the analysis).

Dimension	Groups (tasks)	Dimension	Groups (tasks)
Operation	read 494 · create 232 · update 197 · search/filter 37 · delete 30	Specification	official 545 · partial 445
Authentication	none 495 · Bearer 248 · API key 154 · OAuth2 93	Difficulty (heuristic)	easy 267 · medium 720 · hard 3

Difficulty is the dataset's explicit heuristic (number of selected parameters and fields, and whether a credential is needed), not a measured difficulty; with three "hard" tasks it is reported only descriptively.

The API totals match Progress Report Two exactly, but the operation, difficulty and authentication proportions do not match its approximate targets (for example 233 POST tasks against a target of 150, and 30 DELETE tasks against 100). We chose not to relabel tasks to hit percentages: the generator produces the operations each specification genuinely supports, and the audit documents the difference (DATASET_AUDIT.md). The analysis therefore reports results by subgroup rather than assuming a balanced design.

4.5 The answer key and its checks

The task file is the answer key. Before any model output was scored we checked that the task and prompt files agree, that the prompts are exactly those the benchmark sent (their hash is recorded in every run manifest), that all pinned sources match their hashes, and that expected values follow from the instructions and test inputs. The evaluator's **self-check** executed all 1,000 reference solutions in its sandbox: all were classified correct and all met the behaviour requirements. The planned two-reviewer human review of a stratified 100-task sample of the answer key was not completed (Section 10.2); the review of the cases the curated specifications left open (Section 7.5) partly compensates for it.

4.6 What the benchmark measures, and what it does not

Every task is answerable and has exactly specified ground truth; there are no trick premises, no ambiguous items and no tasks where refusal is the correct answer. The benchmark therefore measures the **natural** rate at which the model writes hallucinated API calls on ordinary requests, which is what a developer would experience, and version drift enters naturally where an API has changed (Petstore, REST Countries). The cost of this choice is that the benchmark cannot reward a correct refusal of an impossible request; we return to this in Section 10.

4.7 Pilot tasks

Ten tasks, one or two per API, were used to develop and debug the agent. They overlap the main dataset and are therefore excluded from every summary statistic; results including them are reported as a sensitivity check (Section 9.3).

5 Baseline Evaluation

5.1 One pinned model configuration

The lecturer's first concern was that earlier documents described the local model and its settings inconsistently (for example Qwen3.6 9B in Progress Report One and Qwen3.5 9B-Instruct in Progress Report Two, with different temperatures and context lengths). The final experiment uses exactly one model, served exactly one way, with the settings in Table 5; both conditions and all three quantization runs use them. The only factor varied between runs is the quantization of the same weights, which we treat as three replications of the experiment rather than as a treatment (Section 7.1).

Table 5**:** The pinned model and run configuration (identical for baseline and intervention; read from the run manifests).

Setting	Value
Model	Qwen3.5-9B (publisher qwen, architecture qwen35, GGUF), loaded in LM Studio as `qwen3.5-9b-q4`, `-q6`, `-q8`
Quantizations	Q4_K_M, Q6_K and Q8_0 (as reported by LM Studio for the loaded model)
Serving software	LM Studio, OpenAI-compatible server on 127.0.0.1:1234; tool use enabled
Context length	30,208 tokens loaded (model maximum 262,144)
Sampling	temperature 0.2; no fixed seed (one sample per task per run); reasoning ("thinking") off
Limits	max 4,096 completion tokens per reply; max 4 model turns per intervention task; 900 s request timeout
Baseline system prompt	"You are an API coding assistant. Write the Python code the user asks for, using the `requests` library. Reply with the complete code in a single fenced python code block."
Intervention system prompt	Confident Agent prompt for toolbox 3.0.0 (SHA-256 `0ebc4dce`…), operator note removed; tool offered: `verify_candidate` only
Prompts sent	rest_api_prompts_v1_DRAFT.jsonl (SHA-256 `c4c68a64`…), identical in both conditions
Software versions	benchmark 2026-09-27.2; toolbox 3.0.0 (decision policy 2026-09-25.1); Pyright 1.1.414; harness Python 3.13.15; Node.js 24.15.0
Machine	Windows 11; AMD Ryzen 7 5700X3D (8 cores); 32 GB RAM; AMD Radeon RX 9070 XT; Durban Central
Run dates (SAST)	Q6_K 30 Sep–1 Oct; Q8_0 1–2 Oct; Q4_K_M 2–3 Oct 2026

5.2 Baseline results

Table 6 gives the class of the code the plain model delivered for each of the 990 scored tasks in each run. Between half and 55% of answers contained an API hallucination; only a quarter to a third were correct.

Table 6**:** Baseline: class of the code delivered to the user (990 tasks per run, pilot excluded).

Class	Q4_K_M	Q6_K	Q8_0
Correct	247 (24.9%)	365 (36.9%)	341 (34.4%)
A: API hallucination	513 (51.8%)	503 (50.8%)	546 (55.2%)
B: valid call, wrong task	79 (8.0%)	56 (5.7%)	58 (5.9%)

Class	Q4_K_M	Q6_K	Q8_0
Runtime failure	150 (15.2%)	63 (6.4%)	43 (4.3%)
No code block	1 (0.1%)	3 (0.3%)	2 (0.2%)

Two features stand out. First, the API hallucination rate is almost the same at every quantization (50.8–55.2%): it is a property of what the model knows, not of numerical precision. Second, the lower quantization pays for its smaller size in ordinary programming errors rather than in API errors. At Q4_K_M, 128 of the 150 runtime failures used an input as the wrong type, typically treating the credential string as a dictionary, and 52 class B answers sent a malformed credential; at Q8_0 these fell to 36 and 7. The API knowledge is shared; the coding care is not.

5.3 Why the baseline fails: the model writes the API it was trained on

Most baseline hallucinations (1,303 of 1,562 across the three runs) are of a single kind: the wrong host, version or base path. Figure 2 shows which host and version every baseline answer called, compared with the frozen specification the task pins.

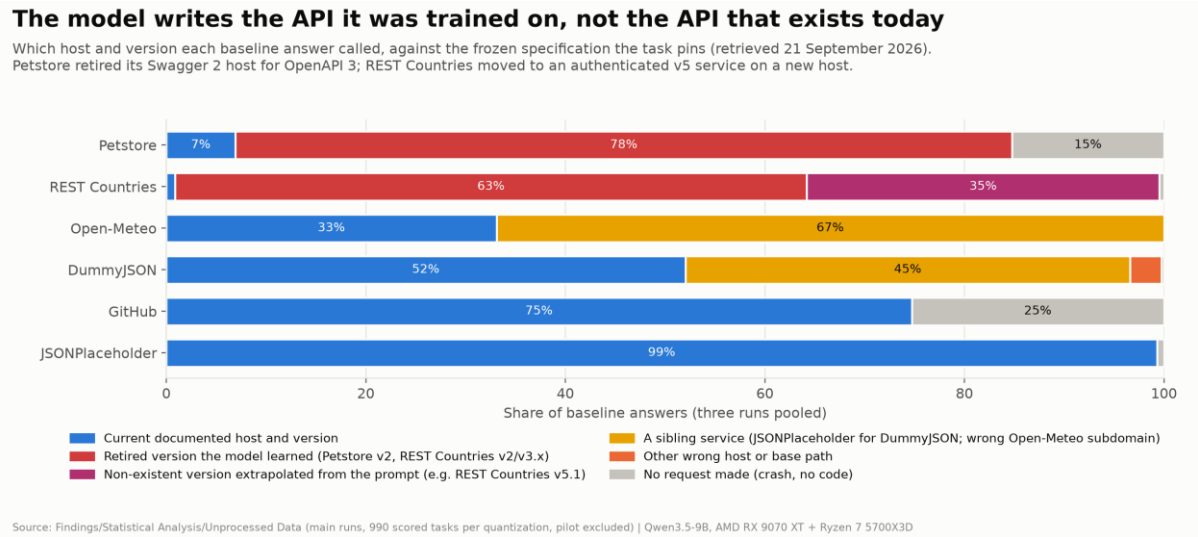


Figure 2: Which API host and version each baseline answer called (three runs pooled). The model reproduces versions from its training data and extrapolates versions it has never seen.

- **Retired versions.** 78% of Petstore answers called the Swagger 2 host `petstore.swagger.io/v2`, which dominates tutorials and training data, instead of the OpenAPI 3 service `petstore3.swagger.io/api/v3` that the benchmark pins. 63% of REST Countries answers called the deprecated `restcountries.com/v3.1` (or older) interface.
- **Extrapolated versions.** The prompt says "REST Countries v5". The model has never seen v5, so in 35% of answers it grafted the number onto the old URL scheme and invented `restcountries.com/v5.1/...`. Only 4 of 447 answers reached the real host `api.restcountries.com/countries/v5`. This is a hallucination created by combining stale knowledge with a fresh cue.
- **Sibling confusion.** 45% of DummyJSON answers called JSONPlaceholder, a different fake-data API with similar resources, and 67% of Open-Meteo answers sent air-quality or marine requests to the forecast host.

- **Where the model is current, it is right.** JSONPlaceholder and GitHub hosts were almost always correct; GitHub errors were mostly crashes, not wrong hosts.

These are exactly the errors a model cannot detect by introspection: nothing in its weights says that Petstore moved. They are also exactly the errors a frozen specification detects deterministically. The Confident Agent prompt names the hosts for which the toolbox holds evidence, which is part of the intervention by design (Section 6.3); Section 8.4 separates how much of the improvement that context explains from what verification adds.

6 Confident Agent Architecture and Design

6.1 Design principles

The lecturer's pattern notes describe six candidate architectural patterns for trustworthy agents (Vorster, 2026a). The Confident Agent instantiates all six; Table 7 shows where. The organising principle is the one the notes state: generation alone must not confer authority on an output.

Table 7: How the Confident Agent realises the Trust-by-Design patterns.

Pattern	Realisation in the Confident Agent
Independent Verification	The model proposes code; a deterministic tool (claim extractor, Pyright, OpenAPI matcher) judges it. The verifier never asks the model whether it is right.
Domain-Specific Validation	Evidence is the API's own contract: OpenAPI paths, parameters, schemas and security schemes, plus the Python package registry and type checker.
Evidence Fusion	Six layer verdicts, each with an evidence tier, are combined by the EWCS; missing and contradictory evidence are kept apart (unverifiable versus verified incorrect).
Confidence Gating	The decision policy releases code only if no layer is contradicted, no critical check is unresolved, and the EWCS clears a threshold; it can deliver, flag, request a correction, or abstain.
Deferred Commitment	A contradicted candidate is not released; the agent may rewrite it once and must have the new code verified. An abstention without a contradiction is final.
Provenance Preservation	Every verification records the candidate's SHA-256, the evidence file and its hash, per-layer evidence details and the decision; the report renderer refuses code whose hash differs from the verified candidate.

6.2 Architecture overview

Figure 3 shows the deployed components. The model runs in LM Studio. The toolbox is a TypeScript package with a small Python claim extractor; it is loaded either as an LM Studio tools provider, for interactive use, or, in the experiment, by a Node.js bridge that the benchmark harness calls whenever the model requests the tool. Both paths execute the same `verify_candidate` implementation. The evaluator is a separate Python program that shares no code with the toolbox.

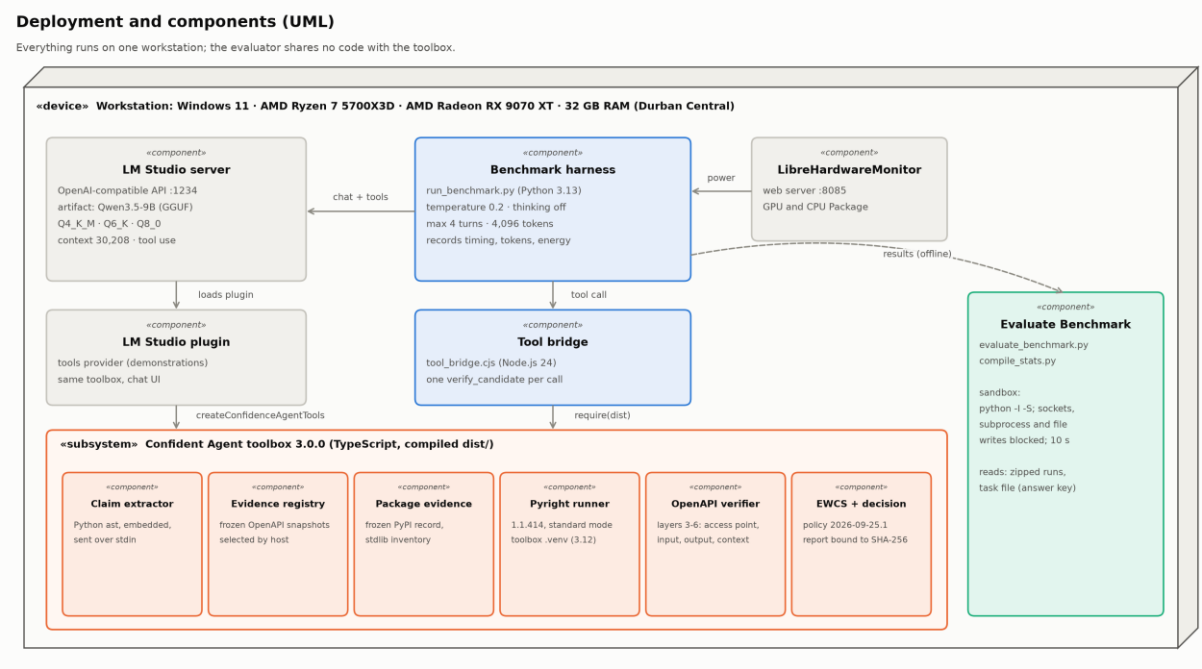


Figure 3: Deployment and component view (UML). All components run on one workstation; the evaluator is independent of the toolbox.

6.3 Where the tool attaches, and why

The most consequential design decision after choosing the domain was where verification attaches to the model. We considered four options (Table 8).

Table 8: Options for attaching verification to the model.

Option	Advantages	Disadvantages
A. External post-hoc controller (CLI): generate, then verify and correct outside the model	Full enforcement; deterministic; the model cannot skip a step	Not how developers use assistants; corrections need a second prompt the model did not plan for
B. In-loop tool via function calling (chosen)	The deployable artefact: a model with a prompt and a tool, as in LM Studio; the model sees the evidence and can correct itself	The model controls the protocol and may deviate; deviations must be measured
C. MCP server (Progress Report Two)	Portable across MCP clients	Adds a server process; LM Studio's tools-provider SDK gives the same function-calling interface natively
D. Multi-agent pipeline (Progress Report One)	Specialist agents per check	Each agent is another model that can hallucinate; deterministic checks are cheaper and stronger evidence

We chose **B** for the experiment because the intervention we want to evaluate is the package a developer would actually run: the same model, a system prompt, and one tool. The trade-off is that the model, not the tool, controls the conversation; it may skip verification, verify twice, or paraphrase the report. We addressed this in three ways. The tool, not the model, makes the delivery decision, and the benchmark scores the tool's decision on the last candidate it verified; code in the model's reply that the tool did not approve is recorded separately (it never occurred). The report embeds the candidate's hash, so a model cannot attach an approving report to different code. And the benchmark

records every protocol deviation (Section 8.8). Option A survives as the toolbox's external controller and pipeline scripts, which we used for engineering checks of the verifier. Option C would be a packaging change, not a design change, and is left for future work. Option D was rejected on evidence grounds: the Trust-by-Design notes caution against using a model as judge where a deterministic checker exists (Vorster, 2026a).

6.4 Claim extraction

Verification starts by turning code into claims. The extractor parses the candidate with Python's `ast` module (it never runs it) and abstractly interprets the function body: it follows variables, f-strings, concatenation, dictionaries built key by key, `if`/`try` branches and helper functions that return `response.json()`. Values that only exist at run time, such as `inputs["organization"]`, become typed holes, so a URL like `f"https://api.github.com/orgs/{org}"` is kept as a template and matched against OpenAPI path templates. Each claim records its type (import, method call, endpoint, input field, output field), its line and its provenance. When the extractor meets a construct it cannot follow (a `requests.Session`, another HTTP client, a class, async code), it reports an issue that names the layers it affects, and only those layers are downgraded to unverifiable; early versions downgraded every layer and silently scored most valid code as unverifiable (Section 6.10).

6.5 The six verification layers

Figure 4 shows how a candidate's claims are distributed over the six layers, the evidence each consults and its weight in the score.

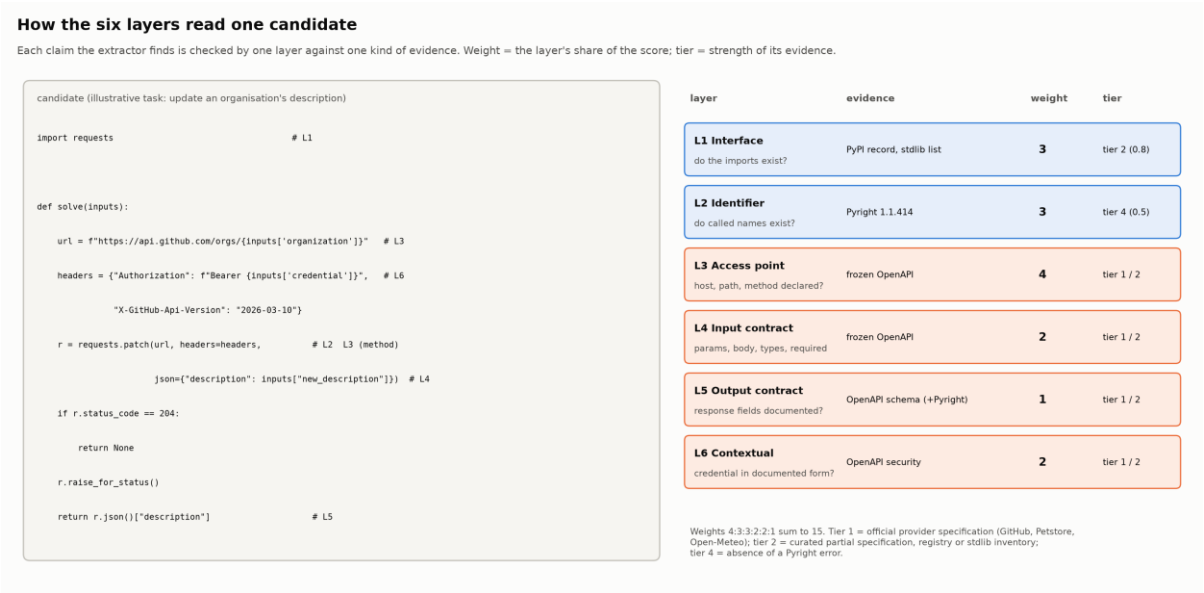


Figure 4**:** How the six layers read one candidate: each claim is checked by one layer against one kind of evidence.

- **L1 Interface.** Every imported module must exist: standard-library modules against the pinned interpreter's module list, third-party packages against a PyPI record (frozen for `requests` in the experiment so that a network timeout cannot change a verdict).
- **L2 Identifier.** Pyright 1.1.414 in standard mode type-checks the exact candidate. An undefined name, attribute or call is a contradiction; argument-type errors count only on API lines. Absence of an error is weak evidence (tier 4).

- **L3 Access point.** The host and base path must match a server of the frozen specification, and the path template and HTTP method must match a declared operation. A mismatch is a contradiction whose evidence names the closest declared operations, which the agent can use to correct. An unknown host has no evidence and makes the layer unverifiable.
- **L4 Input contract.** Required parameters, their locations (path, query, header, body), types and request-body properties are checked against the matched operation. Runtime values are not guessed; an undeclared query parameter or body field is unverifiable unless the schema explicitly allows or forbids extra properties.
- **L5 Output contract.** Every response field the code reads is followed through the success-response schema. A field the schema documents verifies; a field it does not document cannot be verified, and contradicts a schema that excludes it.
- **L6 Contextual.** The operation's security requirements are checked against the actual request: a missing credential or a wrong scheme is a contradiction; OAuth2 scopes, which are granted to a token rather than expressed in code, are reported as a note.

Each layer returns exactly one verdict (verified correct, verified incorrect, unverifiable or not applicable) with an evidence tier and a human-readable evidence detail; a layer aggregates its claims by taking the most severe verdict, so one contradicted parameter contradicts the layer.

6.6 Frozen evidence, selected by host

An early design discovered specifications dynamically at run time, by probing well-known paths and searching the web. We abandoned it for two reasons recorded in our design discussions and confirmed by the lecturer's feedback: a live specification can change between the baseline and agent runs, which destroys the comparison, and four of the six benchmark APIs publish no discoverable specification, so a dynamic resolver would have measured discovery rather than verification. The toolbox instead bundles the same frozen snapshots as the dataset, each with its provenance and hash, and a registry maps each host to its snapshot. The tool selects evidence from the host the candidate itself calls; the model never supplies or summarises a specification. A host without frozen evidence yields no evidence, so layers 3 to 6 are unverifiable and the tool abstains: an invented host cannot be "verified".

6.7 The Evidence-Weighted Confidence Score

Definition

The EWCS answers one question: *what proportion of the applicable verification weight is supported by positive evidence?* Let A be the set of layers that are applicable to the candidate (all six, minus those that are not applicable), w_l the weight of layer l, t_l the factor of the evidence tier that supports its verdict, and c_l = 1 if the layer is verified correct and 0 otherwise. Then

$$EWCS = \frac{\sum_{l \in A} w_l \cdot t_l \cdot c_l}{\sum_{l \in A} w_l}$$

The EWCS lies in [0, 1]. It is 1 only when every applicable layer is verified by authoritative (tier 1) evidence, and it falls both when evidence is weaker and when layers are unverifiable or contradicted. Not-applicable layers leave the denominator, so a candidate is not penalised for having no response fields to read. A diagnostic **signed score** S keeps contradictions visible by using σ_l = +1 (verified correct), −1 (verified incorrect) or 0 (otherwise), so that S lies in [−1, 1]:

$$S = \frac{\sum_{l \in A} w_l \cdot t_l \cdot \sigma_l}{\sum_{l \in A} w_l}$$

The EWCS is **not a probability of correctness**. A score of 0.85 means that 85% of the applicable weight is supported by evidence, not that 85% of such candidates are correct; Section 8.6 shows how far the two differ in practice.

Why these weights

The weights are a design hypothesis, fixed before any run and then tested (Section 8.7). They come from a two-step procedure that is standard in multi-criteria decision analysis when only the *order* of importance can be justified: rank the criteria, then convert ranks to weights with the **rank-sum rule** (Stillwell, Seaver and Edwards, 1981; Barron and Barrett, 1996):

$$w_k \propto n - r_k + 1, \; n = 4 \; severity \; ranks \; \Rightarrow \; 4 : 3 : 2 : 1$$

We ranked the layers by the consequence of a hallucination in that layer for the user, judged on two criteria: whether the call can still do anything useful, and whether the failure is silent or loud. Table 9 gives the ranking and the resulting weights.

Table 9**:** Severity ranking of the layers and the rank-sum weights (total 15).

Rank	Layer	Weight	Why
1	L3 Access point	4	A wrong host, path or method sends the request to a resource that does not exist or to the wrong one; nothing downstream can be right. It is the dominant hallucination kind in our pilot and baseline, and layers 4–6 are only defined relative to the matched operation.
2	L1 Interface, L2 Identifier	3	A non-existent module or function makes the program fail outright. Fatal, but loud: the developer sees an error at once.
3	L4 Input contract, L6 Contextual	2	A wrong parameter or credential usually produces an HTTP error or a partial result. Serious but situational, and frequently loud (4xx responses).
4	L5 Output contract	1	Reading an undocumented field is a downstream data bug; the request itself succeeds.

Why these tier factors

The tier factor discounts a verdict by the strength of the evidence behind it, on the familiar principle of evidence hierarchies that primary, authoritative sources outrank derived ones and that an absence of contrary evidence is weaker than positive evidence (Table 10).

Table 10**:** Evidence tiers and factors.

Tier	Factor	Sources	Rationale
1	1.0	Provider-published OpenAPI (GitHub, Petstore, Open-Meteo)	The provider's own contract: the strongest evidence available without calling the service.
2	0.8	Curated partial specifications; PyPI record; interpreter module list	One step removed from the authority (transcribed from documentation, or registry metadata); a 20% discount reflects transcription and coverage risk.
3	0.9	Runtime execution (reserved, not used)	Observed behaviour would be stronger than metadata but weaker than the contract it may not exercise. The agent never executes code, so tier 3 never occurs.

Tier	Factor	Sources	Rationale
4	0.5	Pyright found no error	Absence of a type error is "no evidence against", not evidence for: half credit, the point of indifference.
none	0	No evidence	No evidence earns no credit.

Worked example and the ceilings it implies

For the GitHub candidate of Figure 4, assume all claims verify: L1 by the PyPI record (tier 2: 3 × 0.8 = 2.4), L2 by a clean Pyright run (tier 4: 3 × 0.5 = 1.5), L3, L4 and L5 by GitHub's official specification (tier 1: 4 + 2 + 1 = 7). GitHub's OpenAPI document does not model authentication, so L6 is not applicable and the denominator is 3 + 3 + 4 + 2 + 1 = 13. Hence EWCS = 10.9 / 13 = **0.838**, and the decision is *deliver with flag*. If the same code had called the invented path `/organizations/{org}`, L3 would be verified incorrect, L4 and L5 could not be matched to an operation (unverifiable), and EWCS = 3.9 / 13 = **0.30**; the score is irrelevant, however, because the contradiction triggers the safety constraint. This is why hallucinated candidates cluster at an EWCS of about 0.3 in Figure 15.

Because the weights and tiers are fixed, the highest EWCS each kind of evidence can reach is known in advance (Table 11). The *deliver* threshold of 0.85 was set at the point that requires authoritative evidence for every API layer *including* authentication: only fully modelled official specifications can reach it. Every candidate checked against a curated specification can at most be *delivered with a flag*, which is the honest outcome for researcher-authored evidence.

Table 11**:** Maximum attainable EWCS when every applicable layer verifies (L1 tier 2, L2 tier 4).

Evidence for layers 3–6	Applicable weight	Max EWCS	Best decision
Official, all of L3–L6 applicable (e.g. Petstore with security)	15	0.860	deliver
Official, no response fields read (L5 n/a)	14	0.850	deliver
Official, authentication not modelled (GitHub, Open-Meteo)	13	0.838	deliver with flag
Official, L5 and L6 n/a	12	0.825	deliver with flag
Curated partial specification (JSONPlaceholder, DummyJSON, REST Countries)	12–15	0.725–0.740	deliver with flag
No API call (only L1 and L2 applicable)	6	0.650	deliver with flag

6.8 The decision policy

Figure 5 gives the policy. It has three ordered rules, and the first two are deliberately independent of the score:

1. **Safety constraint.** If any layer is verified incorrect, the code is not released. On the first candidate the decision is *correct*: the agent may rewrite the code once and must submit the new code for verification. On a second candidate it is *abstain*.
2. **Uncertainty gate.** If a layer failed to run, extraction was incomplete for a layer, or any applicable layer is unverifiable, the decision is *abstain*. A high score built on the remaining layers cannot outweigh an unresolved critical check.

3. **Thresholds.** Otherwise EWCS ≥ 0.85 delivers, 0.60 ≤ EWCS < 0.85 delivers with a flag, and anything lower abstains.

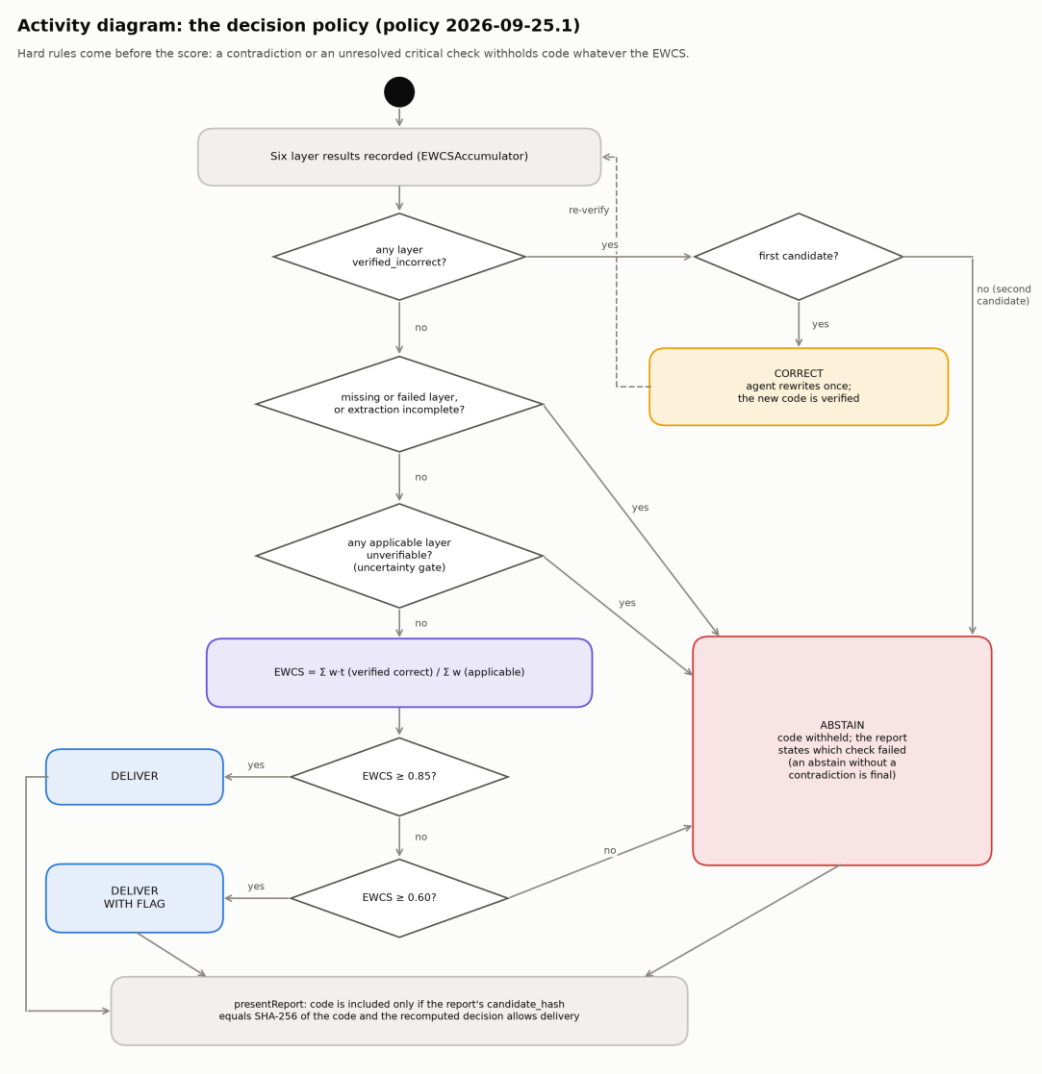


Figure 5**:** Activity diagram (UML) of the decision policy. Hard rules precede the score; the report renderer releases code only when its hash matches the verified candidate.

Two consequences matter for interpreting the results. Because the gate requires every applicable layer to be verified, any candidate that passes it already has an EWCS of at least 0.65 (Table 11); the 0.60 threshold is a backstop that never binds in practice. And because withholding is decided by the safety constraint and the gate, the weights and thresholds only decide *whether delivered code carries a flag*. Section 8.7 confirms both empirically. The correction limit of one follows Progress Report Two: more attempts would let sampling luck masquerade as correction and inflate cost. Finally, the renderer recomputes the decision from the per-layer contributions instead of trusting a score passed to it, and includes code only if the report's candidate hash equals the SHA-256 of that code.

6.9 Static and dynamic structure

Figure 6 shows the main types of the verification core and Figure 7 the interaction for one benchmark task.

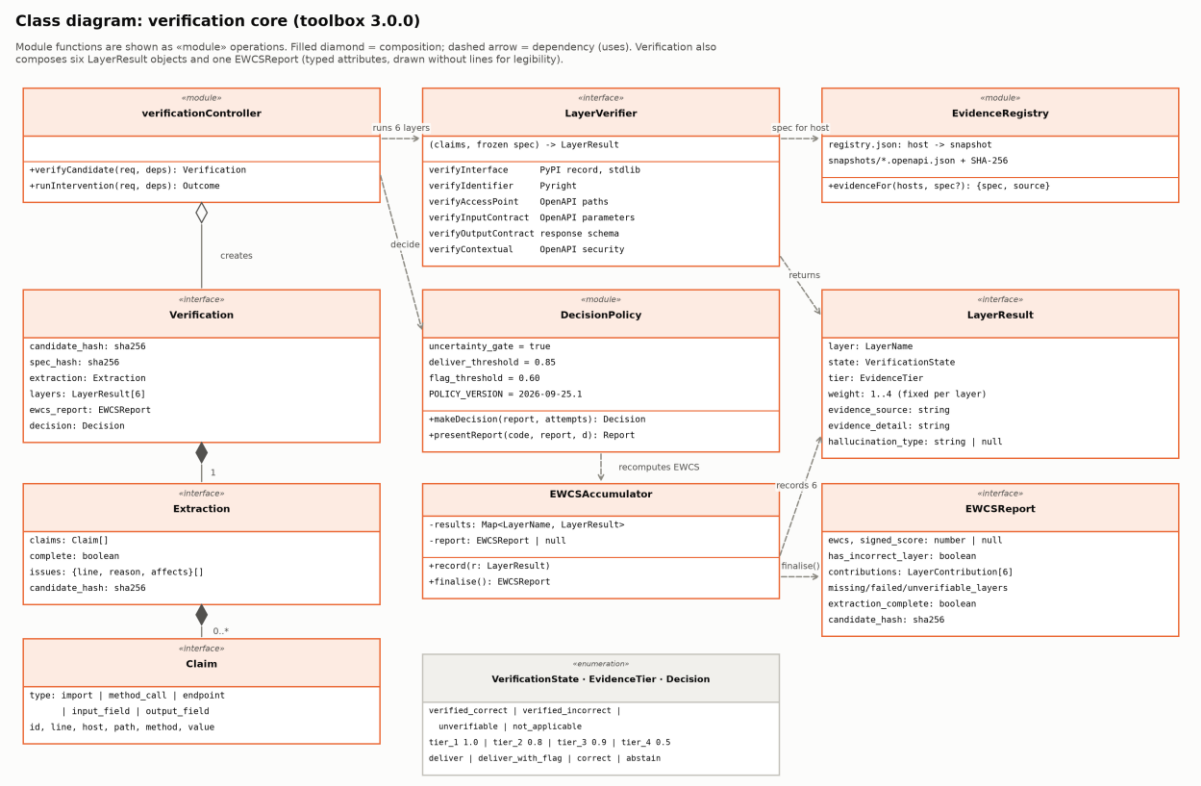


Figure 6: Class diagram (UML) of the verification core in toolbox 3.0.0.

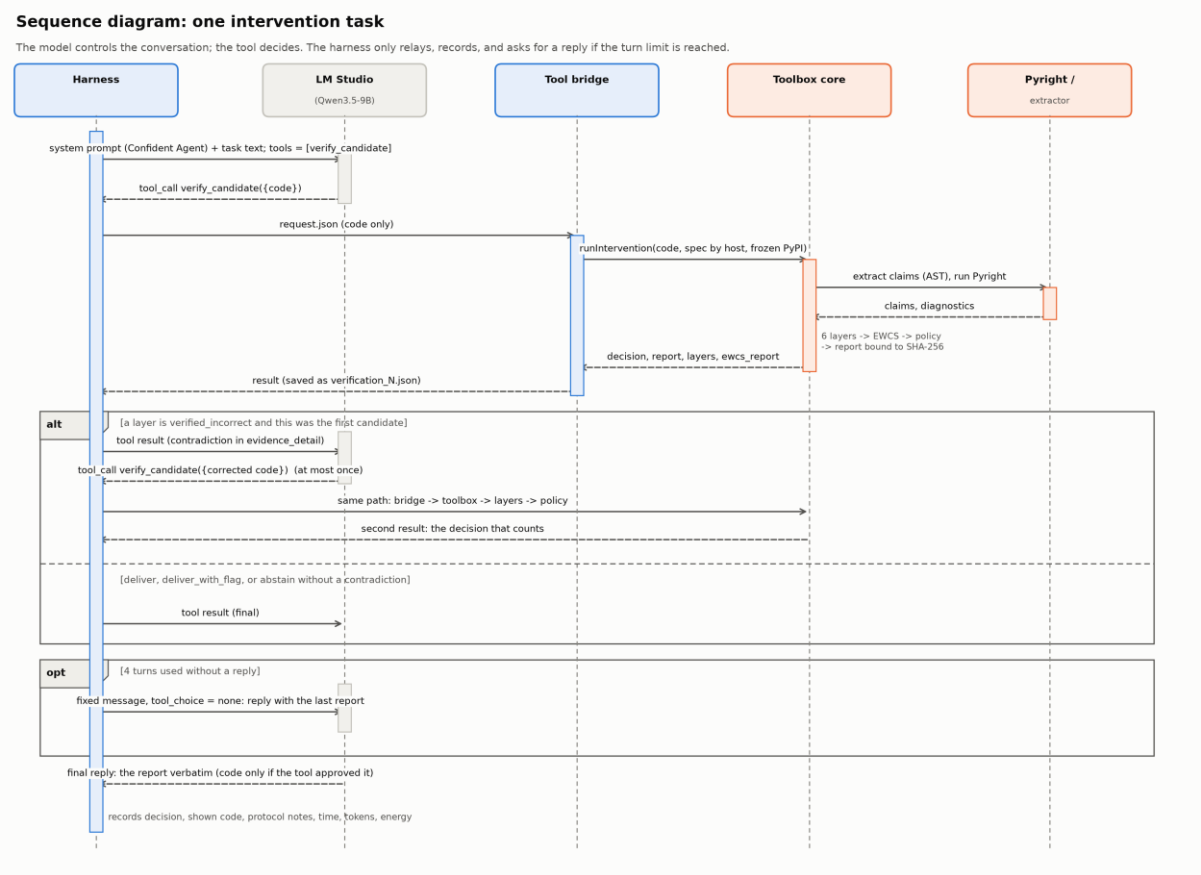


Figure 7: Sequence diagram (UML) of one intervention task, including the single permitted correction and the reply request at the turn limit.

6.10 Design evolution and verification of the verifier

The design changed substantially between Progress Report Two and the experiment, and each change was driven by evidence rather than preference. Three changes stand out.

- **From dynamic discovery to frozen evidence** (Section 6.6): live specifications would have made the comparison irreproducible and the coverage of four APIs zero.
- **From a general-purpose plugin to a standalone toolbox.** The first working version was built into an existing LM Studio plugin that also offered code execution, a terminal, web search, a browser and memory. Those tools would have let the model work around verification and confounded the experiment, and code execution contradicts our safety principle. Toolbox 3.0.0 removes all of them and exposes verification tools only.
- **Thirteen scoring defects found by testing the verifier on known answers.** Running the first implementation on the dataset's own reference solutions (known-correct code) gave a mean EWCS of 0.007 and falsely contradicted valid REST Countries code. The causes were engineering defects, not design flaws: any unsupported construct downgraded every layer, `headers["X"] = v` assignments were invisible, URLs containing a runtime value became wholly unknown, Pyright ran in strict mode, and others. All thirteen were fixed and documented with the decisions they required.

Table 12 summarises the engineering checks of the final verifier. These are checks of the apparatus, not experimental results: they show that it accepts known-valid code and rejects known-invalid code before it was used on model output.

Table 12**:** Engineering checks of the verifier on known-valid and known-invalid code (toolbox reference regression).

Input	Result
1,000 reference solutions (known valid), before the fixes (150-task sample)	mean EWCS 0.007; 138 abstain, 12 false contradictions
1,000 reference solutions, final verifier	mean EWCS 0.784; 99 deliver, 888 deliver with flag, 13 abstain; **0 contradictions**
1,000 mutants with an invented path	all 1,000 contradicted (correction requested)
1,000 mutants with an undeclared method	all 1,000 contradicted
1,000 mutants with a wrong host	all 1,000 abstain (no evidence for an unknown host)

The 13 remaining abstentions on valid code are documented evidence limits (GitHub request bodies declared with `oneOf`, one ambiguous path template).

One change was made after inspecting benchmark run 1, and is disclosed as such in the evaluation plan: an undeclared request-body field is now unverifiable, as an undeclared query parameter already was, because eight run-1 deliveries had sent hallucinated body fields (for example `name` where GitHub's label API declares `new_name`). All findings in this report use the verifier after this change.

7 Experimental Methodology

7.1 Design

The experiment is a **paired, within-task comparison**. Every one of the 1,000 tasks is answered twice by the same model with the same sampling settings: once in the baseline condition and once in the intervention condition. The unit of analysis is the task; the primary outcome is binary per task and condition. The whole experiment was run three times, at three quantizations of the same model, which gives three independent replications with almost 3,000 scored task pairs. Quantization differences are described, not tested, because each quantization was run once (Figure 8).

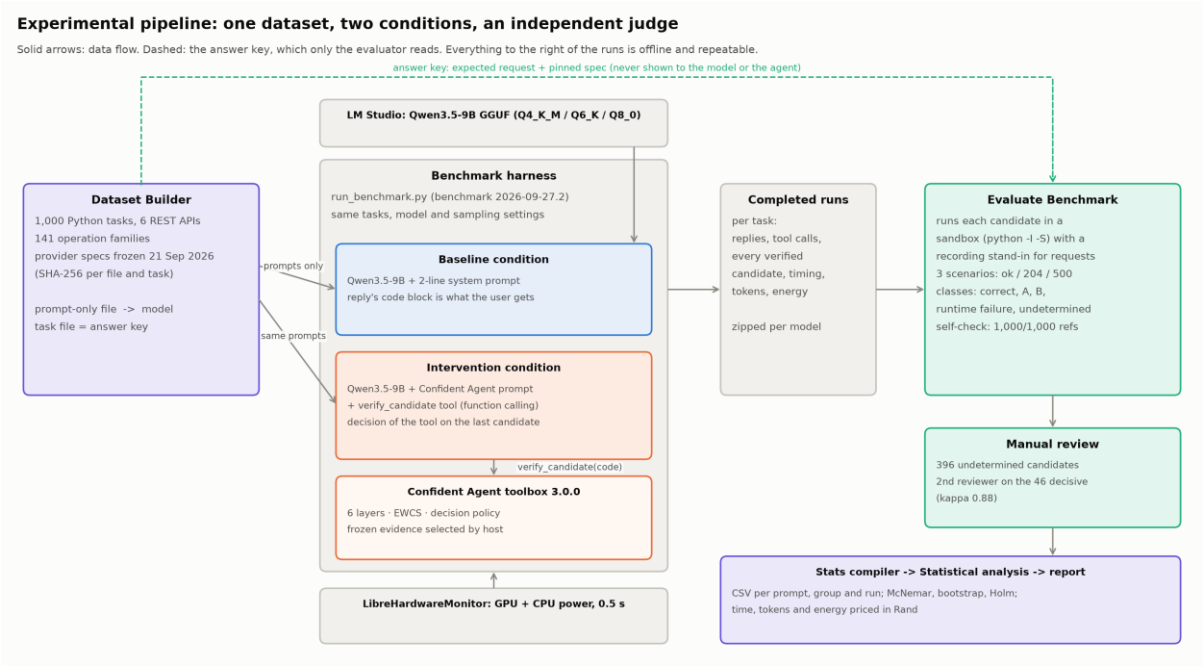


Figure 8: Experimental pipeline. The answer key reaches only the evaluator; the model and the agent see the prompt alone.

Each condition generates its own code. The dataset audit suggested reusing a single initial answer for both conditions, which would isolate the verifier. We chose not to, because the intervention under test is the deployable package of prompt and tool, whose prompt also shapes the first answer. The consequence, that some of the improvement comes from the prompt's context rather than from verification, is quantified in Section 8.4.

7.2 Procedure

The benchmark harness sends each prompt to LM Studio. In the **baseline**, the model's single reply is stored and the Python code block it contains is what the user would receive. In the **intervention**, the model may call `verify_candidate` with its code; the harness executes the call through the tool bridge, stores the candidate byte for byte with the tool's full result, and returns the result to the model. The prompt instructs the model to correct once if, and only if, a layer is verified incorrect, and then to reply with the tool's report verbatim. If the model has not replied after four turns, the harness sends one fixed message with tools disabled asking for the report. What reaches the user is the code the tool delivered on the last verified candidate, or code shown in the model's reply; the latter would

be recorded as unapproved (it never occurred). Every message, tool call, candidate and timing is saved, and tasks are resumable.

7.3 Measurements

Table 13**:** What was measured for every task and condition.

Measure	Definition
Primary outcome	Class A code reaches the user (1/0), as classified by the independent evaluator
Secondary outcomes	class A or B; any error (A, B or runtime failure); correct code reaches the user; incorrect acceptance and rejection; abstention
Verifier behaviour	decision, EWCS, layer states, corrections, protocol deviations, verification calls, tool failures
Task behaviour	returns the expected value; returns None for an empty 204 response; raises on a 500 error
Time	wall-clock seconds per task, split into model time (LM Studio) and tool time (verification)
Tokens	prompt and completion tokens summed over the task's requests, as reported by LM Studio
Energy	GPU package and CPU package power from LibreHardwareMonitor, sampled every 0.5 s and integrated over the task; idle power measured for 10 s before each run

AMD GPUs offer no command-line power reading on Windows, so the harness reads LibreHardwareMonitor's local web server, running as administrator, for the "GPU Package" sensor of the RX 9070 XT and the CPU "Package" sensor. The measured energy excludes memory, storage, motherboard and power-supply losses, so the true wall-socket energy is higher. Energy is converted to Rand at the eThekwini Municipality tariff on the electricity account for the test site in Durban Central for September 2026, **R3.5747 per kWh excluding VAT**, plus 15% VAT, i.e. **R4.1109 per kWh** (eThekwini Municipality, 2026):

$$Cost\ (R)\ =\ \frac{E\ (J)}{3\ 600\ 000\ J/kWh}\ \times\ R3.5747/kWh\ \times\ 1.15\ =\ E\ \times\ 1.1419\ \times\ 10^{-6}\ R/J$$

7.4 Independent evaluation

The lecturer stressed that the experimental grader must not trust the agent's own verdict (Vorster, 2026c). The evaluator therefore shares no code with the toolbox and uses a different method (Figure 9). It executes each candidate in a separate, isolated Python process (`python -I -S`) with the task's test inputs and a recording stand-in for `requests` that has the real library's names and signatures but sends nothing: sockets, subprocesses and file writes are blocked and each run has a 10-second limit. This yields the exact requests the code would send, including those built in loops and branches, and every response field it reads. Each candidate runs in three scenarios (a documented success response built from the response schema, an empty 204 response, and a 500 error), which give the behaviour checks. The requests are then compared with the expected request and the pinned specification and classified as in Table 1.

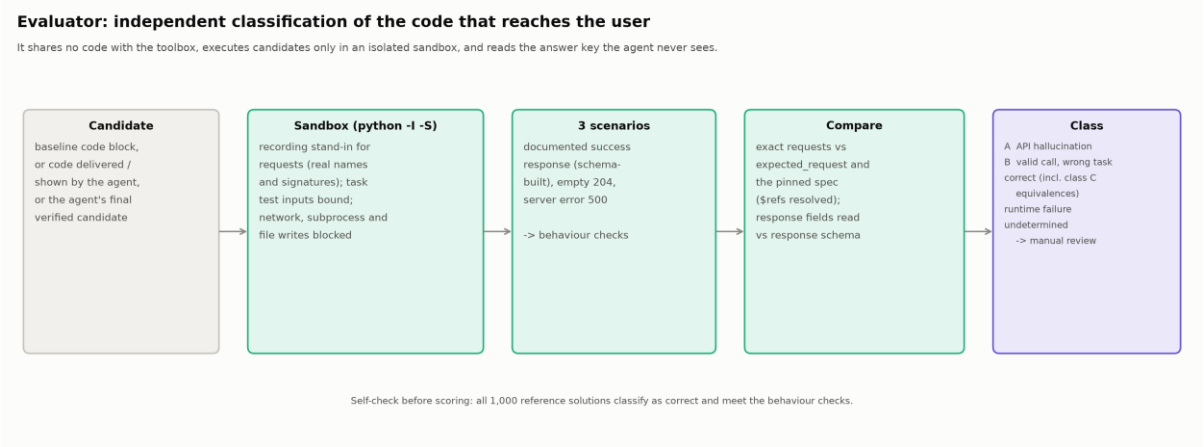


Figure 9: The independent evaluator: sandboxed execution, three response scenarios and comparison with the answer key.

Before scoring any model output, the evaluator scored all 1,000 reference solutions: every one was classified correct and met all behaviour checks. This self-check guards against an evaluator that is stricter or looser than the answer key.

7.5 Manual review of undetermined candidates

For the three curated specifications, a name the specification does not list cannot be classified automatically. The evaluation plan required such candidates to be resolved by manual review against the providers' documentation. Across the three final runs there were **396 distinct undetermined candidates**: 46 had reached the user and could change the primary result, and 350 had been withheld by the agent and affect only the verifier's confusion matrix.

Each open question (for example: does DummyJSON support a `sort` query parameter? does REST Countries v5 have a `/v5/name/{name}` path?) was checked once against the provider's documentation and, where the documentation was silent, against the live public API with harmless read requests (for example, `?sort=title` returns exactly the unsorted list, while the documented `?sortBy=title` sorts it). Each answer was recorded as an explicit rule, so that every candidate asking the same question received the same verdict, and the evaluation plan's rules were then applied unchanged: a name the provider does not have is class A; a name it has is judged as for an official specification. The first review was made with AI assistance. A group member then independently reviewed the 46 decisive candidates: the reviewers agreed on 43 of 46 (Cohen's $\kappa = 0.88$). The three disagreements were JSONPlaceholder candidates that read a resource from a list endpoint (`/comments?id=…` instead of `/comments/{id}`); the group settled them as correct because they return the requested resource, which relaxes the closed equivalence list for those three cases and changes no hallucination figure. All 396 decisions, with their reasons, are in the submission (`reviews.csv`, `review_rules.py`).

7.6 Pre-registration and deviations

To avoid choosing scoring rules after seeing results, the scoring rules, outcomes and statistical tests were fixed in an evaluation plan on 27 September 2026, before benchmark run 2. Every later change was logged with its reason (Table 14).

Table 14**:** Changes to the evaluation plan after it was fixed.

Date	Change	Reason and effect
27 Sep	Undeclared request-body fields become unverifiable in the verifier	Found by inspecting run 1 (8 hallucinated fields delivered); disclosed; applies to all reported runs
30 Sep	Holm family limited to the main runs	Decided before those runs were scored
6 Oct	Findings use the final series (Q4_K_M, Q6_K, Q8_0, 30 Sep–3 Oct) instead of run 2 and the first Q6_K/Q8_0 runs	Machine throughput halved mid-way through the first Q6_K run (GPU power 240 W to 193 W), invalidating its cost figures; accuracy agrees with the first series within about 2 points (Section 8.11)
6 Oct	Manual review made with AI assistance, not by two independent members	Time constraints; documented rules and reasons for all 396 decisions
7 Oct	Second, human review of the 46 decisive candidates; three relaxed to correct	κ = 0.88; changes correct-code figures by one task (Q6_K) and two (Q8_0)

7.7 Statistical analysis plan

Table 15**:** Statistical methods (evaluation plan §6).

Question	Method
Does the intervention change the primary outcome?	Exact two-sided McNemar test on the paired binary outcome (McNemar, 1947); α = 0.05
How large is the change?	Difference in proportions with 95% percentile bootstrap intervals (10,000 resamples) over tasks and, to allow for related prompts, over the 141 operation families (Efron and Tibshirani, 1993)
Three runs, one hypothesis	Holm's step-down correction across the three runs (Holm, 1979)
Per-group rates	Wilson 95% intervals (Wilson, 1927); subgroup tests are descriptive
Cost	Wilcoxon signed-rank test on paired per-task costs (Wilcoxon, 1945) with rank-biserial effect size; Kruskal–Wallis across quantizations
Confidence metric	Area under the ROC curve, correct versus class A final candidates; share correct per EWCS band
Agreement	Cohen's κ for the two reviewers and between repeated runs (Cohen, 1960)
Robustness	Pilot tasks included; undetermined counted as A and as correct; hallucinations other than wrong host; the five planned weighting schemes

8 Experimental Results and Comparative Analysis

All figures in this section are computed over the 990 scored tasks per run (pilot excluded), after the manual review; no undetermined candidates remain. Supplementary charts are collected on the A3 summary page in Appendix A.

8.1 Headline results

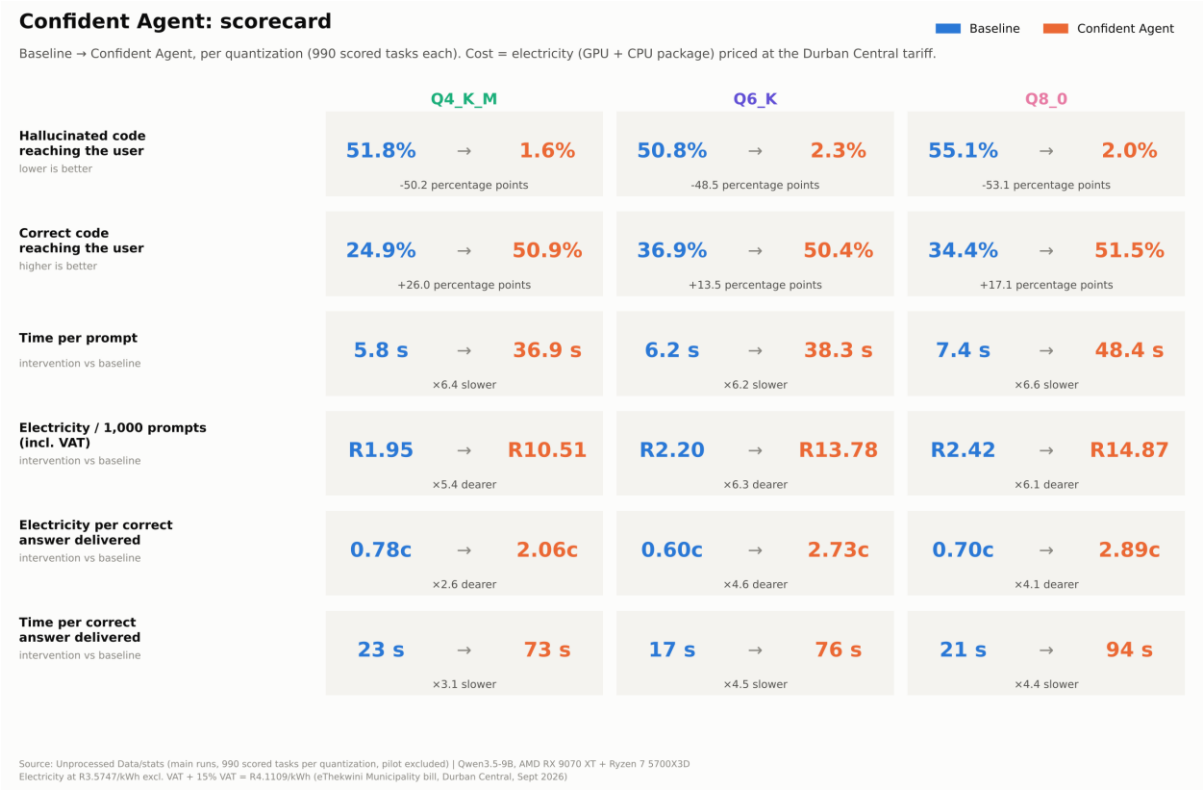


Figure 10**:** Scorecard: baseline versus Confident Agent for each quantization (990 tasks each). Electricity priced at R4.1109/kWh including VAT.

Table 16**:** Headline outcomes per run (990 tasks each).

Outcome	Q4_K_M	Q6_K	Q8_0
Hallucination (A) reaching the user: baseline → agent	51.8% → 1.6%	50.8% → 2.3%	55.2% → 2.0%
Tasks repaired / regressed (A)	505 / 8	490 / 10	529 / 3
Correct code reaching the user: baseline → agent	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
Abstention (nothing shown)	37.6%	39.5%	38.7%
Incorrect acceptance (A delivered)	16	23	20
Incorrect rejection (correct final candidate withheld)	21	20	28
Mean time per prompt: baseline → agent	5.8 → 36.9 s	6.2 → 38.3 s	7.4 → 48.4 s
Electricity per 1,000 prompts (incl. VAT)	R1.95 → R10.51	R2.20 → R13.78	R2.42 → R14.87

Pooled over the three runs, the share of tasks in which hallucinated code reached the user fell from **52.6% (1,562 of 2,970) to 2.0% (59)**, and the share in which correct code reached the user rose from

32.1% (953) to 50.9% (1,513). The reduction is almost identical at every quantization, as is the agent's final accuracy (50.4–51.5% correct), even though the baselines differ: the lower-precision model gains most (+26.0 points of correct code at Q4_K_M) because the agent repairs the coding errors in which Q4_K_M's baseline was weakest. The price is that 38–40% of tasks end with no code at all, and each prompt takes about six times as long.

8.2 What happens to each baseline outcome

Because the design is paired, we can follow every task from its baseline outcome to its intervention outcome (Figure 11). Of the 1,562 tasks in which the baseline delivered a hallucination, the Confident Agent withheld the code in **1,001**, delivered correct code in **427**, delivered a valid call that did not match the task in 91, and still delivered a hallucination in only **38**. Of the 262 baseline crashes, 165 became correct code. The flow in the opposite direction is small: of the 953 tasks the baseline answered correctly, the agent delivered correct code for 819, withheld 76, delivered a different valid call for 51, and turned exactly one into a hallucination.

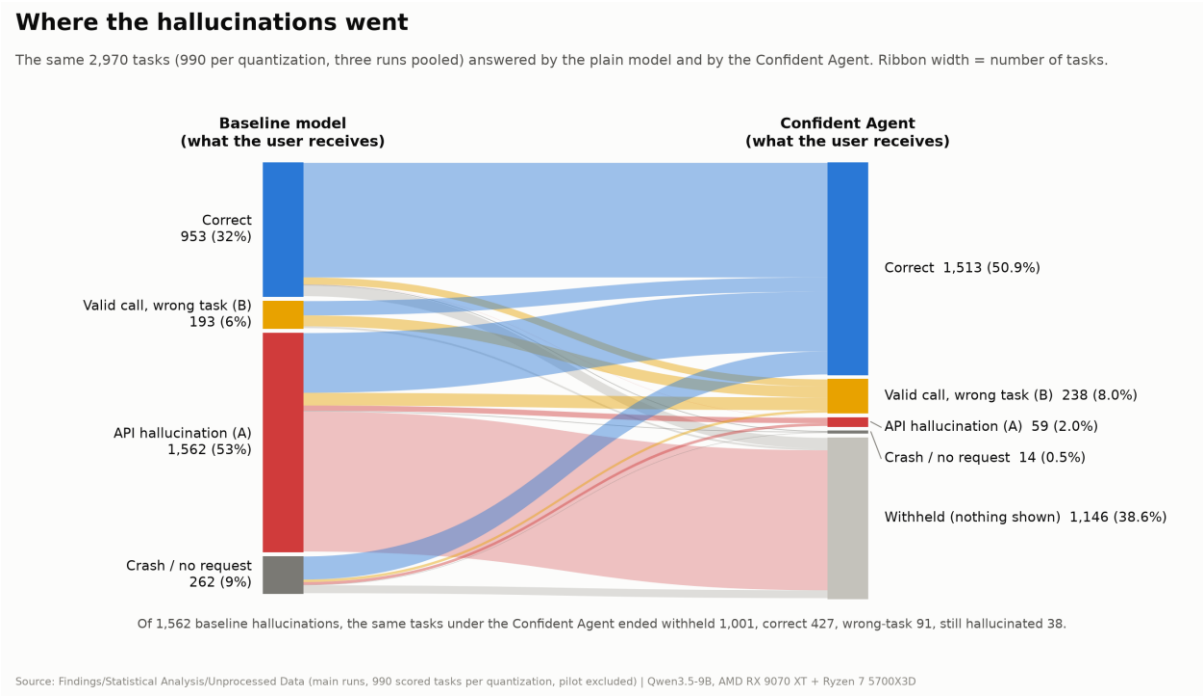


Figure 11: The same 2,970 tasks under the baseline (left) and under the Confident Agent (right); ribbon width is the number of tasks.

> **Reading the abstentions correctly.** About two-thirds of the hallucinations the agent prevented were prevented by withholding code, and one-third by correcting it. Abstention is the intended behaviour of a Confident Agent when evidence contradicts a candidate, but it is not an answer: for a developer it means "this assistant could not produce verified code for this request". We therefore report correct code delivered (H2) alongside the hallucination rate, and treat the 1,146 withheld tasks as a cost.

8.3 Where it works and where it struggles

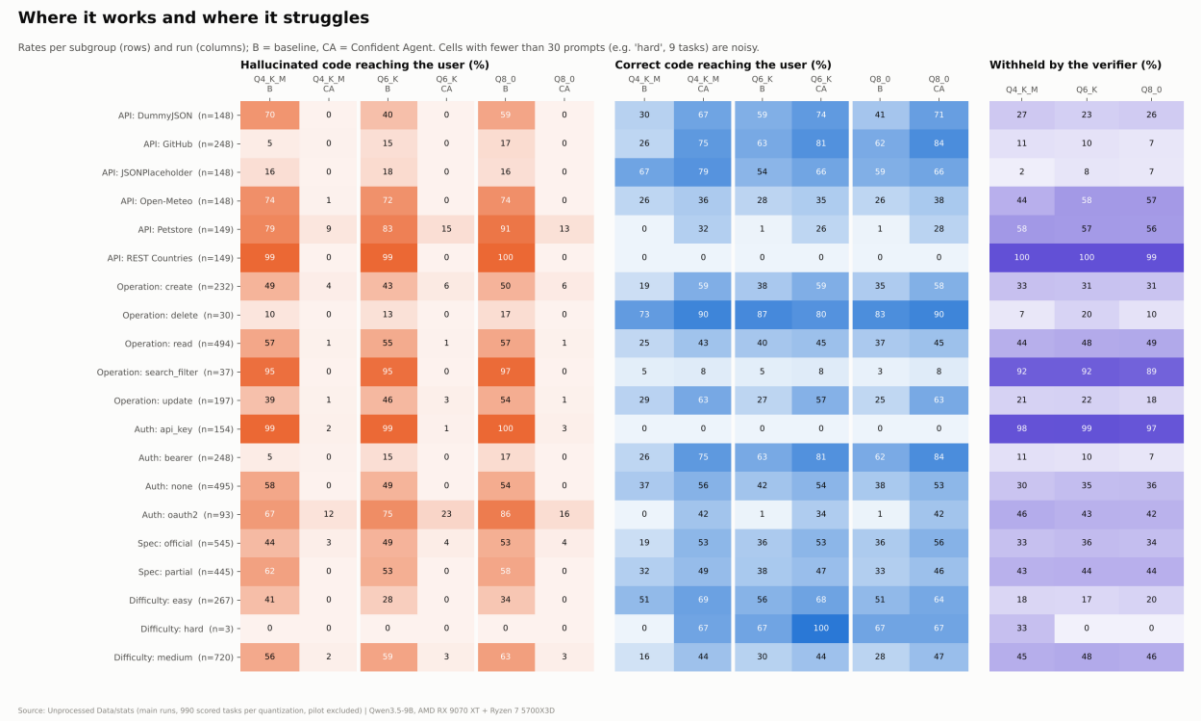


Figure 12**:** Rates per subgroup and run: hallucinated code reaching the user, correct code reaching the user, and code withheld by the verifier (B = baseline, CA = Confident Agent).

Figure 12 breaks the results down by API, operation, authentication, specification source and difficulty. Three patterns dominate.

- **APIs the model knows well improve the most.** GitHub (bearer authentication) rises from 26–63% correct to 75–84%, DummyJSON from 30–59% to 67–74%, with no hallucination reaching the user in any run.
- **Petstore carries almost all the residual hallucinations**: 57 of the 59 hallucinations still delivered across the runs are Petstore tasks (9–15% of Petstore tasks), mostly OAuth2 operations whose request bodies put a run-time value where the schema expects a string (Section 8.4). This is the verifier's main blind spot.
- **REST Countries is entirely withheld.** The baseline hallucinated on 99–100% of REST Countries tasks; the agent delivered nothing for 99–100%. With the current host named in its prompt, the model still invented v5 paths (all 258 withheld candidates reviewed in Section 7.5 used non-existent paths), and the verifier, with only a partial specification, correctly refused them. The result is safe but useless for this API: a correct refusal of the model's attempts, not an answer.

8.4 Kinds of hallucination, and how much is context

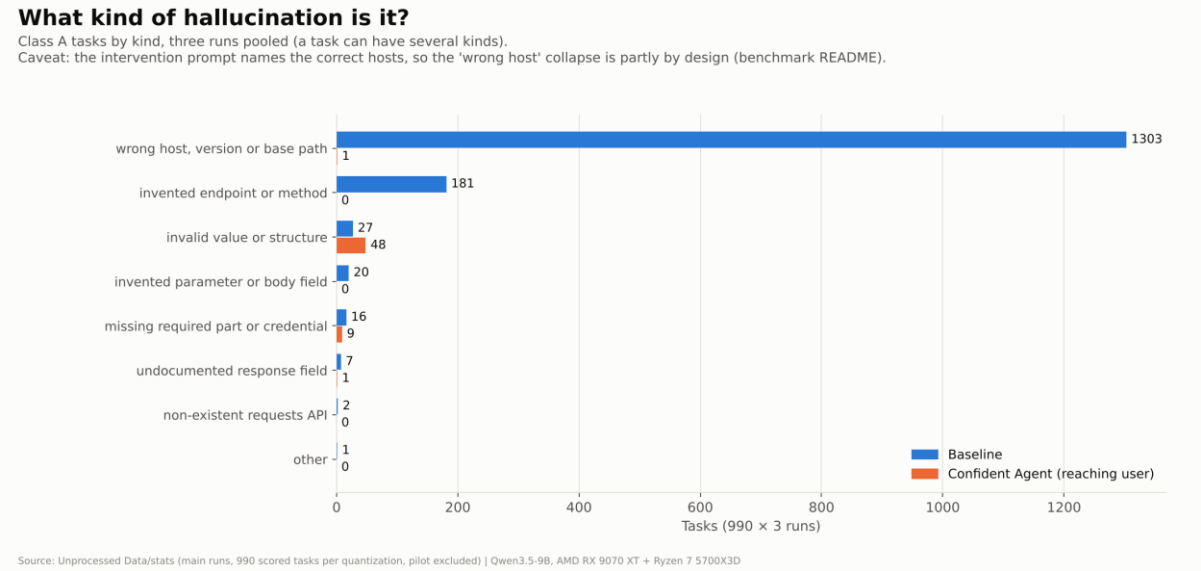


Figure 13**:** Kinds of API hallucination in the code reaching the user (three runs pooled; a task can have several kinds).

Figure 13 shows that wrong hosts, versions and base paths make up most baseline hallucinations, and they almost vanish under the intervention. Part of that is not verification at all: the Confident Agent prompt lists the hosts for which the toolbox holds evidence, which is part of the intervention by design (the model's training data predates the current versions). To separate that context from what verification catches, the evaluator also counts tasks with a hallucination **other than** a wrong host. These fall from 58, 87 and 93 tasks per run to 15, 23 and 20, a reduction of **74–79%** that the host list cannot explain. The REST Countries result points the same way: knowing the right host did not stop the model from inventing the rest of the URL; the specification did.

One kind of hallucination grows: *invalid value or structure*, from 27 to 48 pooled tasks. In 48 of the 59 residual hallucinations a Petstore request body places a value from `inputs` where the schema expects a string (for example `category.name` set to the object `{"id": 1}`), and in 9 more a required `api_key` credential is missing. The cause is a deliberate design decision: the verifier treats values that only exist at run time as opaque, which is what makes `solve(inputs)` code verifiable at all, so it cannot see *which* value lands in a field. The evaluator binds the task's test inputs and can. Binding representative inputs during verification, which the extractor already supports, is the most direct improvement to the design (Section 11).

8.5 The verifier as a hallucination detector

Treating the verifier as a classifier of the agent's final candidate (positive = class A, predicted positive = withheld) gives the confusion matrices of Figure 14. Recall is **94–96%**: it lets very few hallucinations through. Precision is **86–92%**: most of what it withholds really is hallucinated. Its false alarms (20–28 correct candidates withheld per run, plus some wrong-task and crashing candidates) are the incorrect rejections the brief asks us to count. When class B errors are also treated as positives, recall drops to 76–80%: the verifier checks the API's contract, not whether the call is the one the task wanted, so a valid call to the wrong endpoint passes by design.

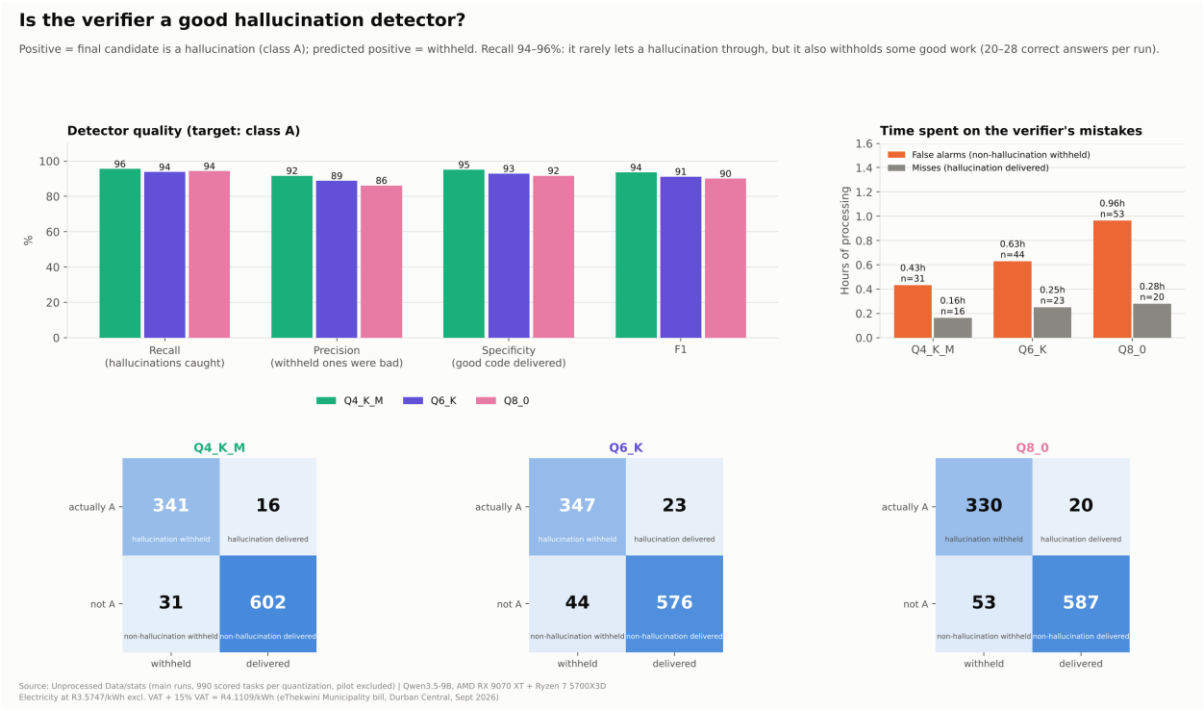


Figure 14**:** The verifier as a detector of hallucinated final candidates: quality measures, confusion matrices, and the processing time spent on its errors.

8.6 The confidence metric

H3 holds: the EWCS separates correct from hallucinated final candidates with an area under the ROC curve of **0.932–0.943** (Figure 15). The distribution explains why. Hallucinated candidates cluster near 0.3, the value left when the access point is contradicted and the API layers that depend on it cannot be matched (Section 6.7); correct candidates cluster at the evidence ceilings of Table 11 (0.74 for curated, 0.825–0.838 for official specifications).

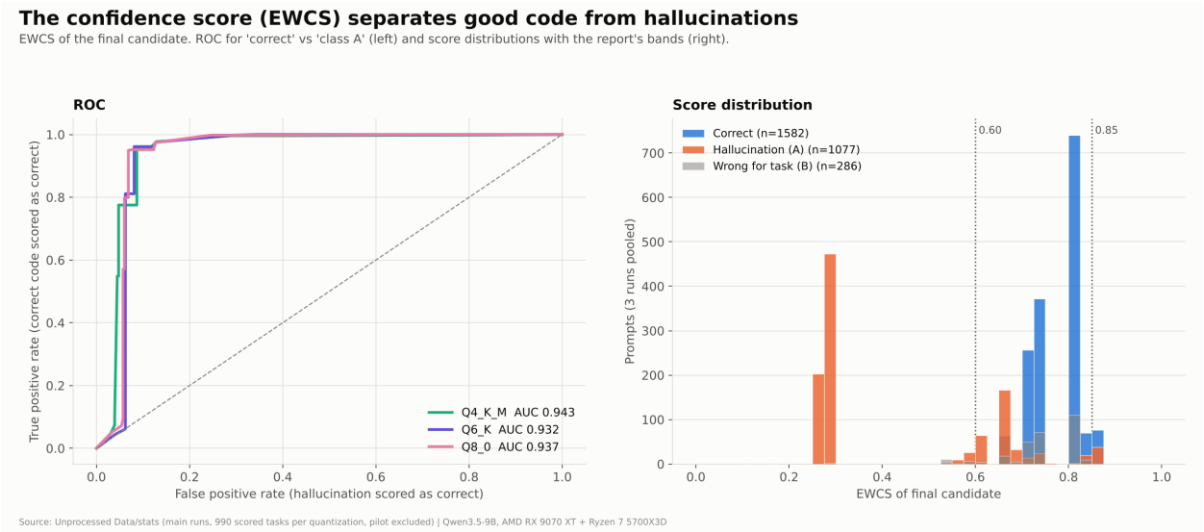


Figure 15**:** EWCS of the final candidate: ROC curve for correct versus class A, and score distributions with the decision thresholds.

Table 17**:** Share of decided final candidates (correct, A or B) that were correct, by EWCS band (three runs pooled).

EWCS band	Candidates	Correct	Share correct
≥ 0.85 (deliver)	166	110	66.3%
0.60–0.85 (deliver with flag)	2,041	1,469	72.0%
< 0.60 (abstain)	738	3	0.4%

Table 17 confirms that the EWCS is not a probability. Candidates scoring 0.85 or more were correct only 66% of the time, *less* often than the flagged band, because the high band is reachable only for Petstore, the API with most residual errors and the most task-fidelity (class B) mistakes. The score measures how much of the claim the evidence covers, not how likely the code is to do what the user asked. The low band, by contrast, is almost pure error: 3 of 738 candidates below 0.60 were correct.

8.7 Sensitivity to the weights, thresholds and gate

The lecturer asked us to treat the six-layer score as a hypothesis, state the weights and thresholds in advance, report all planned schemes, and say whether any result depends on them. Progress Report Two planned five schemes. Because every verification stored its per-layer states and tiers, we re-scored the final candidate of every task under each scheme, plus a sixth that removes the uncertainty gate. The re-scoring is exact: under the scheme actually used it reproduces the tool's recorded decision for all 2,970 tasks. It is also a fair counterfactual, because the agent's only behavioural trigger, a correction, depends on contradictions and not on the score.

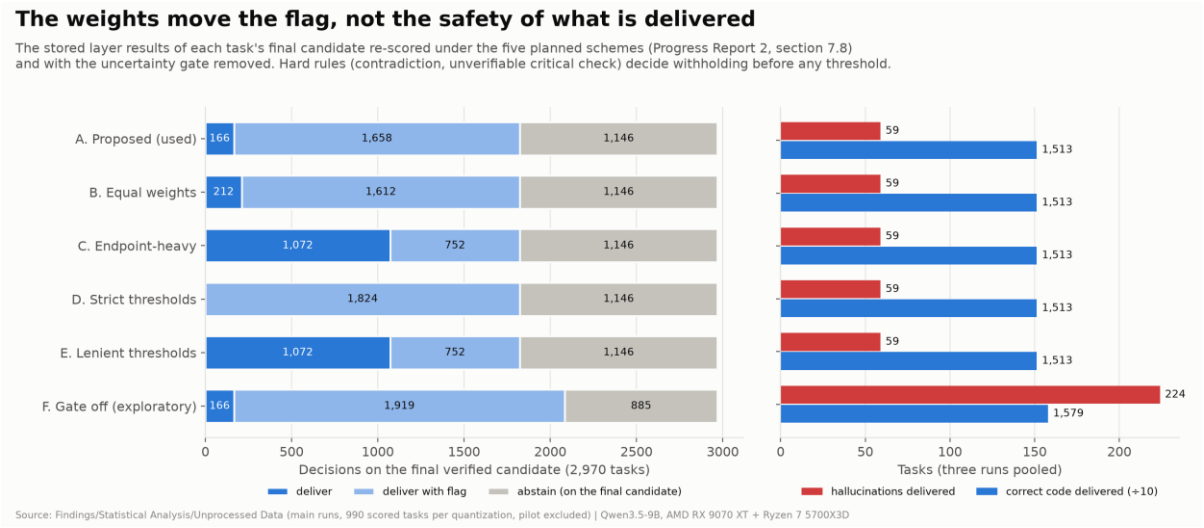


Figure 16**:** Decisions on the final candidate under the five planned schemes and with the uncertainty gate removed (three runs pooled).

Table 18**:** Sensitivity analysis: outcomes on the final verified candidate (three runs pooled, 2,970 tasks).

Scheme	Weights L1–L6	Thresholds	Deliver / flag	Abstain	Hallucinations delivered	Correct delivered
A. Proposed (used)	3,3,4,2,1,2	0.85 / 0.60	166 / 1,658	1,146	59	1,513
B. Equal weights	1,1,1,1,1,1	0.85 / 0.60	212 / 1,612	1,146	59	1,513

Scheme	Weights L1–L6	Thresholds	Deliver / flag	Abstain	Hallucinations delivered	Correct delivered
C. Endpoint-heavy	2,2,6,2,1,2	0.85 / 0.60	1,072 / 752	1,146	59	1,513
D. Strict thresholds	3,3,4,2,1,2	0.90 / 0.70	0 / 1,824	1,146	59	1,513
E. Lenient thresholds	3,3,4,2,1,2	0.80 / 0.50	1,072 / 752	1,146	59	1,513
F. Gate removed (exploratory)	3,3,4,2,1,2	0.85 / 0.60	166 / 1,919	885	**224**	1,579

The answer is unambiguous. **Across all five planned schemes, the set of withheld tasks, the hallucinations delivered and the correct code delivered are identical**; the weights and thresholds only move tasks between *deliver* and *deliver with flag*. The primary result therefore does not depend on any weighting choice. What it does depend on is the uncertainty gate: without it, the agent would deliver 261 more candidates, of which 165 are hallucinations and only 66 correct. The gate converts "I could not verify this" into withholding, and that, together with the safety constraint, is where the protection comes from.

8.8 Correction and agent behaviour

The agent attempted a correction in 403–444 tasks per run, and the prompt permitted it (a layer was verified incorrect) in 349–387 of them. Table 19 shows how the first verified candidate changed by the final one.

Table 19**:** Class of the first verified candidate → class of the final candidate, for tasks in which the code changed.

Transition	Q4_K_M	Q6_K	Q8_0
A → correct (hallucination repaired)	121	123	150
A → A (still hallucinated; withheld)	164	165	166
A → B (valid call, wrong task)	36	33	36
Runtime failure → correct	23	2	0
Corrections attempted / permitted	403 / 349	425 / 377	444 / 387
Final candidate correct after a correction	37.7%	34.1%	37.8%

Self-correction works about a third of the time. When the verifier names the contradiction and the closest declared operation, the model repairs a hallucination in roughly 120–150 tasks per run; about as often it produces another hallucinated candidate, which is then withheld. Figure 17 shows that more correction rounds rarely pay: prompts that needed two or three rounds cost 57–61 s and ended in correct code only 15% and 1% of the time.

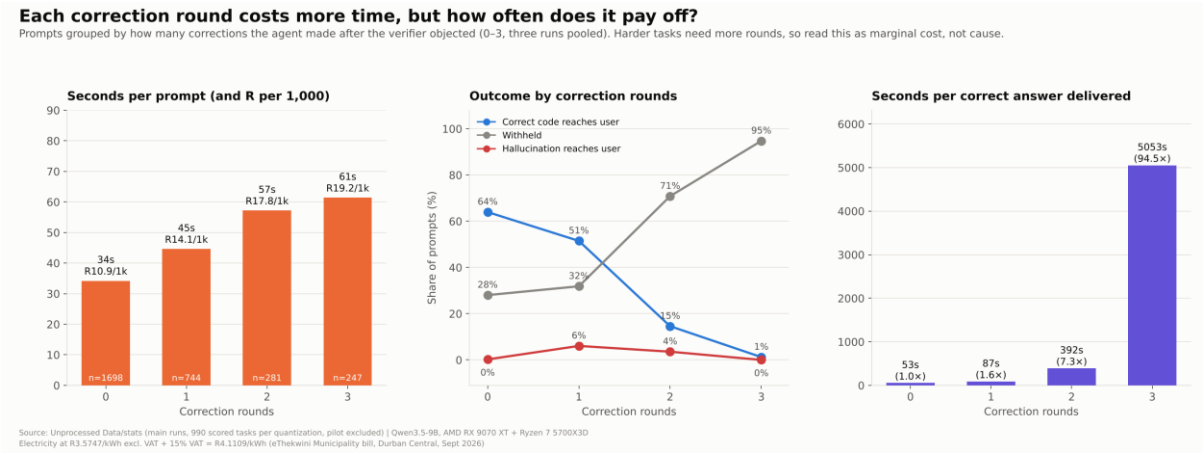


Figure 17**:** Cost and outcome by the number of correction rounds the agent made (three runs pooled).

The model often bent the protocol (Figure 18): 34–39% of tasks reached the four-turn limit and needed the reply request, about 30% re-verified identical code, and 15–19% made more than one correction although the prompt permits one. Only 53–54% of tasks followed the protocol exactly. These deviations cost time, but they never let unverified code through: the tool's decision on the last verified candidate is what reached the user, and **in no task of any run did the user see code the tool had not approved**. This is the practical argument for keeping the delivery decision in the tool rather than in the model (Section 6.3).

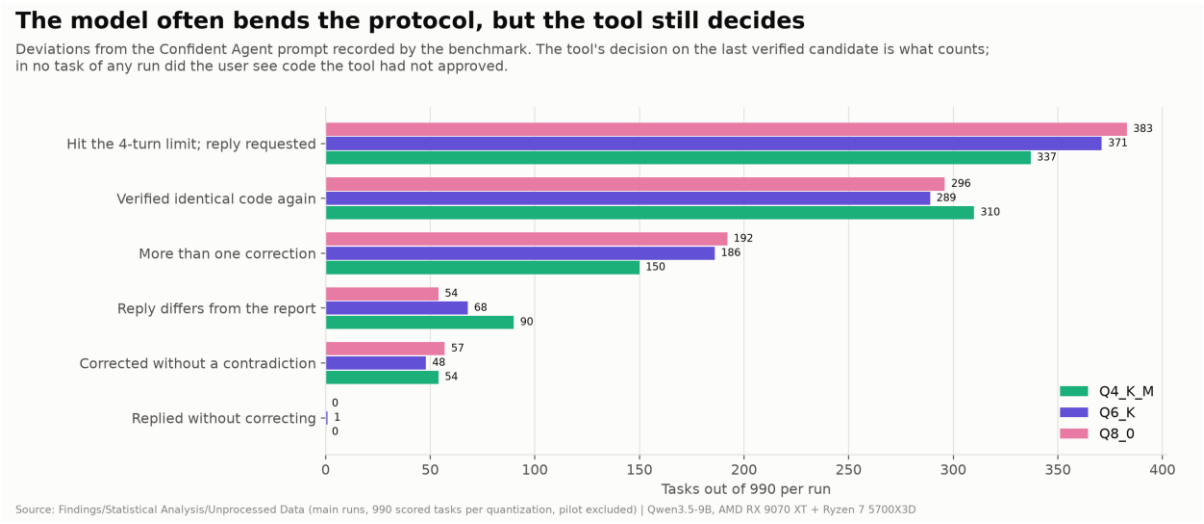


Figure 18**:** Deviations from the Confident Agent prompt recorded by the benchmark, per run.

8.9 Behaviour beyond API correctness

The verifier checks API claims, not whether the code does everything the task asked. The evaluator's behaviour checks show the gap (Table 20). Code reaching the user nearly always raises on a server error and usually returns the expected value, but few programs return `None` for an empty 204 response as instructed, in either condition. Only 19–23% of the agent's delivered programs met all three requirements, against 9–19% of the baseline's. A clean report is evidence about API claims only, which is exactly what the toolbox's report says to the user.

Table 20**:** Task-behaviour checks on code that reached the user (share of programs that make a request).

Requirement	Q4 base	Q4 agent	Q6 base	Q6 agent	Q8 base	Q8 agent
Returns the expected value	80%	83%	86%	82%	86%	84%
Returns None on an empty 204 response	12%	25%	21%	24%	22%	26%
Raises on a 500 error	100%	100%	100%	100%	100%	99%
All three met (all 990 tasks)	8.7%	19.4%	18.5%	22.3%	17.7%	22.5%

8.10 The cost of the intervention

Every prompt costs more under the intervention: in all 18 paired Wilcoxon tests (time, energy, total, generated, GPU and CPU energy, for each quantization) $p < 10^{-160}$, the rank-biserial correlation is at least 0.995, and 99.6–100% of individual prompts cost more (Figure 19). The agent takes **6.2–6.6 times** as long per prompt and uses **5.4–6.3 times** the measured energy.

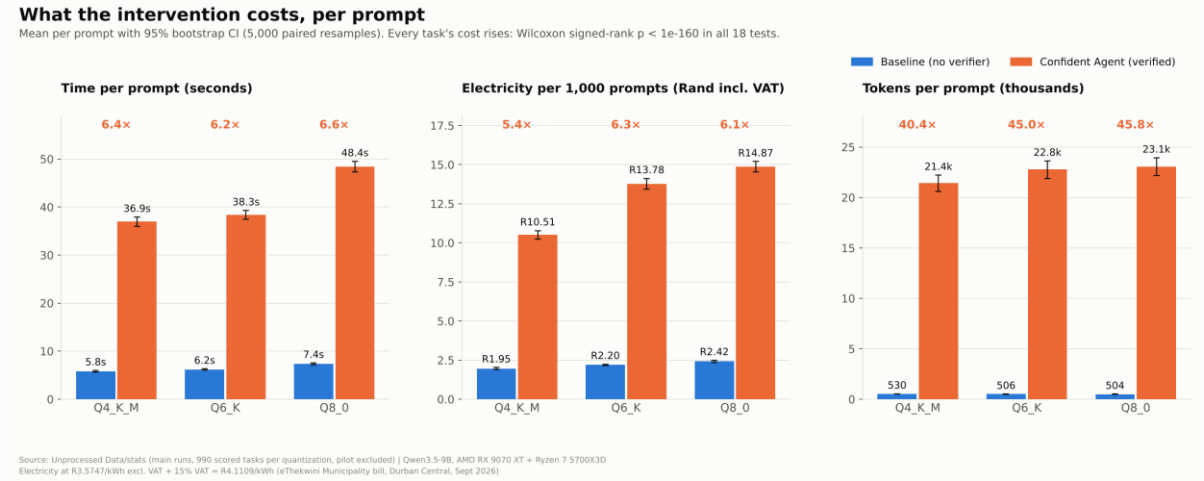


Figure 19**:** Time, electricity (Rand per 1,000 prompts) and tokens per prompt, with 95% bootstrap intervals.

Table 21**:** Cost of the intervention in time, energy and Rand (990 tasks per run; electricity at R4.1109/kWh incl. VAT).

Measure	Q4_K_M	Q6_K	Q8_0
Time per prompt, baseline → agent	5.8 → 36.9 s	6.2 → 38.3 s	7.4 → 48.4 s
Energy per prompt (GPU + CPU)	0.48 → 2.56 Wh	0.53 → 3.35 Wh	0.59 → 3.62 Wh
Electricity per prompt	0.20c → 1.05c	0.22c → 1.38c	0.24c → 1.49c
Tokens per prompt (input + output)	530 → 21,420	506 → 22,771	504 → 23,058
Time per correct answer delivered	23 → 73 s	17 → 76 s	21 → 94 s
Electricity per correct answer delivered	0.78c → 2.06c	0.60c → 2.73c	0.70c → 2.89c
Extra electricity per hallucination prevented	1.70c	2.39c	2.34c
Extra time per hallucination prevented	1.03 min	1.10 min	1.29 min
Whole run, 1,000 prompts: hours	1.6 → 10.2	1.7 → 10.7	2.0 → 13.5
Whole run, 1,000 prompts: Rand	R1.95 → R10.49	R2.19 → R13.79	R2.42 → R14.87

Measured energy covers the GPU and CPU packages only; wall-socket energy is higher. Whole-run figures include the 10 pilot tasks. Across the three runs the verifier added 29.0 hours and R32.59 of electricity (R28.34 before VAT).

In money, the electricity is small: about one cent per verified prompt, two to three cents per correct answer delivered, and about two cents per hallucination prevented. The real price is time and GPU occupancy. At these rates, 10,000 prompts take 0.7–0.9 days on this machine without the verifier and 4.3–5.6 days with it (R20–R24 against R105–R149 of electricity). Figure 20 expresses the cost per good outcome.

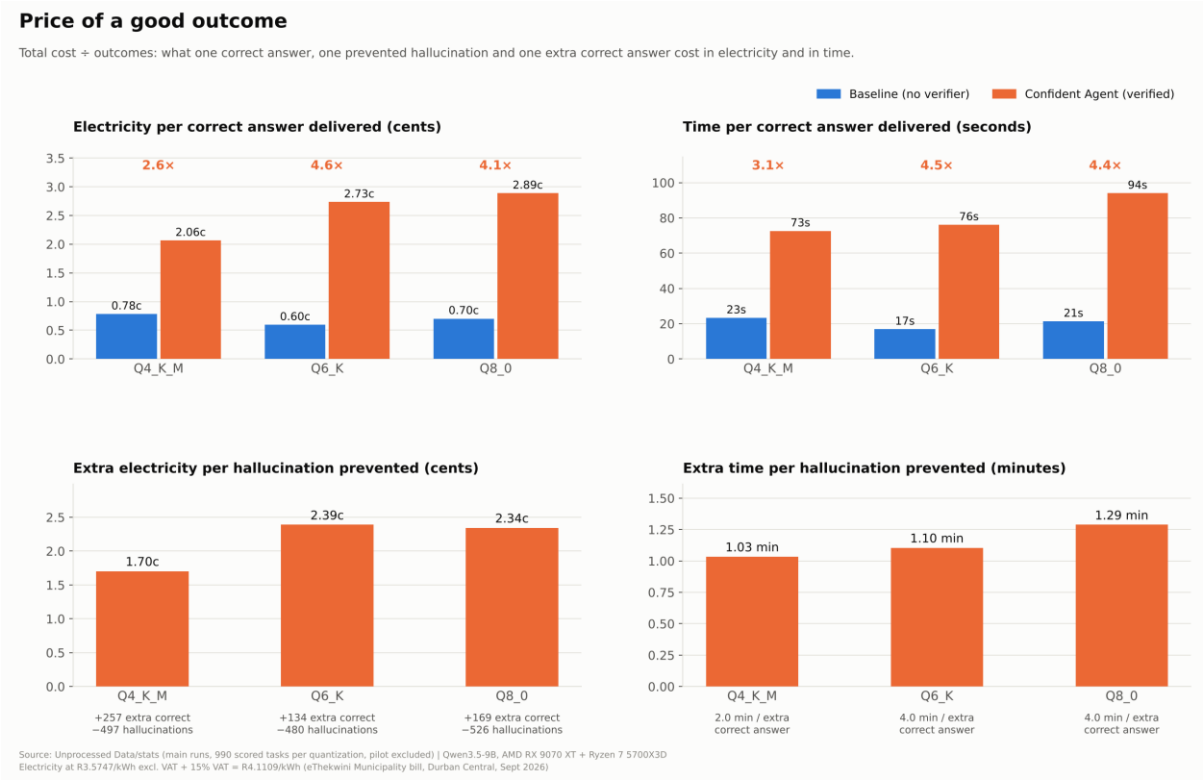


Figure 20**:** What one correct answer, one prevented hallucination and one extra correct answer cost in electricity and time.

Why the cost is so high. The tool itself is cheap: verification takes 2.3–9.6 s per prompt (6–25% of the time). The cost comes from the conversation. Each turn re-sends the whole growing context, including the six-layer report, so a verified prompt processes about 20,000–21,600 input tokens against 156 for the baseline (about 130 times as many) and generates 3.5–4.3 times as many tokens; input length alone tracks the time per prompt (Spearman ρ = 0.92). Energy rises less than tokens because input tokens are cheap to process. The 34–39% of tasks that hit the turn limit pay for the extra reply round as well.

Where the money goes. About 35–38% of the agent's processing time is spent on tasks that end in a correct delivery, and about half on tasks whose final candidate was a hallucination that was then withheld: 5.0–6.2 hours per run (R5.01–R6.65), money well spent from a safety standpoint but buying no deliverable (Figure 21). False alarms cost 0.4–1.0 hours per run. REST Countries alone consumed 7.1 hours and R7.93 across the three runs and delivered nothing.

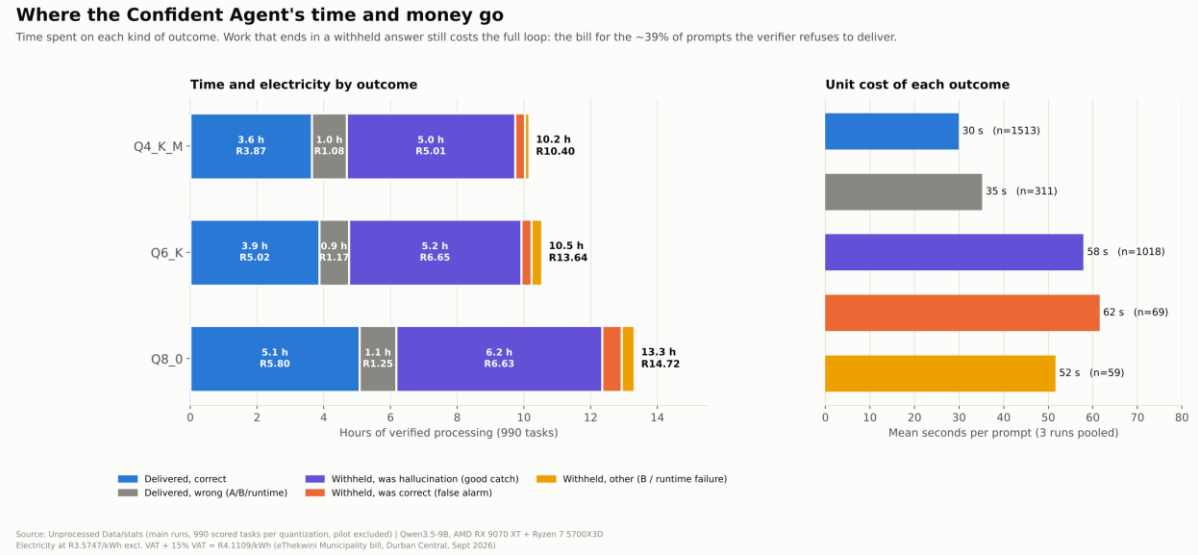


Figure 21**:** Where the Confident Agent's time and electricity go, by outcome.

8.11 Quantization and run-to-run reliability

With the verifier, accuracy is almost independent of quantization (50.4–51.5% correct, 1.6–2.3% hallucinated), so the cheapest model is the best value: Q4_K_M needs 36.9 s and R10.51 per 1,000 prompts against 48.4 s and R14.87 for Q8_0, and delivers 48 correct answers per Rand of electricity against 35. Cost differences between quantizations are statistically clear (Kruskal–Wallis $p < 10^{-49}$ for time and energy in both conditions), but accuracy differences are descriptive: each quantization ran once, on a different day.

Run-to-run variation can be estimated because Q4_K_M and Q8_0 were each run twice with identical settings (the first series was superseded for its cost figures, Section 7.6). Table 22 compares them. The headline rates agree within about two percentage points, but individual tasks vary: at temperature 0.2 without a fixed seed, 11–14% of tasks changed between delivered and withheld from one run to the next. Single-task outcomes are therefore noisy while aggregate rates are stable, which is why the analysis relies on aggregate, paired statistics.

Table 22**:** Repeat runs with identical settings: first series (superseded) versus final series.

Measure	Q4_K_M	Q6_K	Q8_0
Hallucination reaching user, first series	50.8% → 1.6%	51.1% → 2.9%	53.1% → 2.0%
Hallucination reaching user, final series	51.8% → 1.6%	50.8% → 2.3%	55.2% → 2.0%
Correct code reaching user, first series	23.9% → 50.5%	37.1% → 48.8%	36.5% → 50.3%
Correct code reaching user, final series	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
Same baseline class in both series (κ)	76.1% (0.63)	81.7% (0.70)	82.2% (0.70)
Same agent decision in both series (κ)	86.5% (0.75)	87.9% (0.78)	88.8% (0.79)
Tasks whose delivered/withheld status changed	134 (13.5%)	120 (12.1%)	111 (11.2%)

The first Q6_K run was slowed by an environmental fault but its accuracy is unaffected. Agreement statistics were computed before the manual review of the final series.

9 Statistical Analysis

9.1 Primary hypothesis

The primary outcome is paired and binary, so the appropriate test is McNemar's test, which depends only on the discordant pairs: tasks where the baseline hallucinated and the agent did not (b), and the reverse (c) (McNemar, 1947). We use the exact binomial version. Because related prompts share an operation family, we report two 95% intervals for the difference in proportions: a bootstrap over tasks, and a cluster bootstrap over the 141 operation families, which is wider and does not assume tasks are independent.

Table 23**:** Primary test: class A code reaching the user, baseline versus Confident Agent.

Run	Baseline	Agent	Difference	95% CI (tasks)	95% CI (families)	b / c	Holm p
Q4_K_M	51.8%	1.6%	−50.2 pp	[−53.3, −47.0]	[−63.2, −34.6]	505 / 8	1.7×10^{-137}
Q6_K	50.8%	2.3%	−48.5 pp	[−51.7, −45.3]	[−62.1, −32.7]	490 / 10	1.5×10^{-130}
Q8_0	55.2%	2.0%	−53.1 pp	[−56.3, −50.0]	[−65.7, −37.9]	529 / 3	1.1×10^{-152}

H1 is supported in every run. The null hypothesis of no difference is rejected with Holm-adjusted p-values below 10^{-129}, and even the conservative family-level intervals exclude zero by more than 30 percentage points. For every task the intervention made worse, it repaired between 49 and 176.

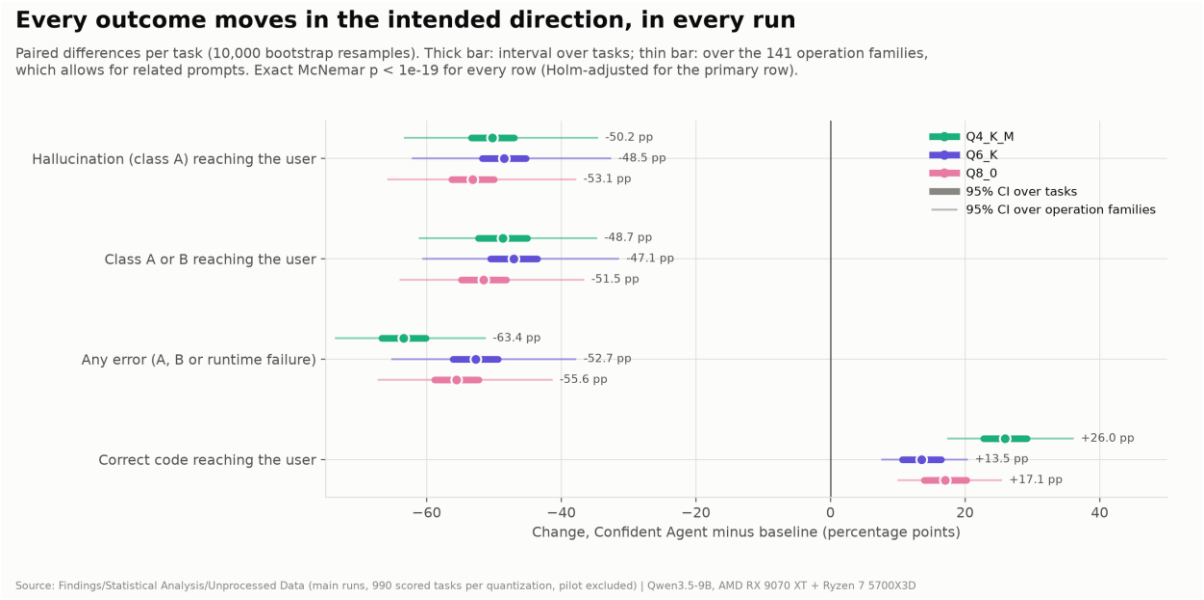


Figure 22**:** Paired differences for the primary and secondary outcomes, with 95% bootstrap intervals over tasks (thick) and operation families (thin).

9.2 Secondary outcomes

Table 24**:** Secondary outcomes (exact McNemar tests; intervals over tasks).

Outcome		Q4_K_M	Q6_K	Q8_0
Class A or B reaching the user	rate	59.8% → 11.1%	56.5% → 9.4%	61.0% → 9.5%

Outcome		Q4_K_M	Q6_K	Q8_0
	p	1.5×10^{-107}	6.1×10^{-112}	4.7×10^{-128}
Any error (A, B or runtime failure)	rate	74.9% → 11.5%	62.8% → 10.1%	65.4% → 9.8%
	p	1.9×10^{-164}	5.4×10^{-134}	2.9×10^{-140}
Correct code reaching the user (H2)	rate	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
	gain [CI]	+26.0 [22.7, 29.2]	+13.5 [10.7, 16.5]	+17.1 [13.9, 20.2]
	p	3.2×10^{-51}	6.4×10^{-20}	5.7×10^{-26}

H2 is supported: correct code reaching the user did not merely survive the intervention, it increased significantly in every run, by 13.5 to 26.0 percentage points (family-level intervals all above +7.7 points). Errors of every class fall, and the reduction in task-fidelity errors (class B) is smaller than in hallucinations, as expected from a verifier that checks the API contract rather than the task.

9.3 Robustness checks

- **Pilot tasks included** (1,000 tasks): 51.8% → 1.6%, 50.7% → 2.3%, 55.0% → 2.0%; all $p < 10^{-130}$.
- **Undetermined candidates**: the plan required the primary result to hold with every undetermined candidate counted as a hallucination and as correct. After the manual review there are none, so the two bounds coincide with the primary row; before the review, the bounds differed by at most 1.5 points and every conclusion held under both.
- **Hallucinations other than a wrong host**: 58/87/93 → 15/23/20 tasks (−74% to −79%), so the result does not rest on the host list in the prompt (Section 8.4).
- **Weighting schemes**: identical withheld sets and outcomes under all five planned schemes (Section 8.7).
- **Repeat runs**: the first series reproduces the headline within about two points (Section 8.11).

9.4 Cost

H4 is quantified rather than tested for direction. Paired Wilcoxon signed-rank tests on time, energy and tokens give $p < 10^{-160}$ in every one of 18 comparisons with rank-biserial correlations of at least 0.995; the ratio-of-means intervals are narrow (for example, time ×6.36 [6.13, 6.58] at Q4_K_M). Quantization affects cost significantly in both conditions (Kruskal–Wallis, Holm-adjusted pairwise tests significant for time and energy), but, with one run per quantization, those differences cannot be separated from day-to-day machine conditions.

9.5 Agreement

Agreement between the AI-assisted first review and the independent second review on the 46 decisive candidates was 43/46, Cohen's $\kappa = 0.88$, which is "almost perfect" agreement on the Landis and Koch (1977) scale. Agreement between repeated runs of the same model and settings was $\kappa = 0.63$–0.70 for the baseline class and 0.75–0.79 for the agent's decision ("substantial"), which quantifies the sampling noise discussed in Section 8.11.

9.6 Interpretation

Put in practical terms, for every 100 requests a developer makes, the plain model hands over about 53 hallucinated API calls and 32 correct programs. The Confident Agent hands over about 2 hallucinated calls and 51 correct programs, says "I could not verify this" about 39 times, and takes about six times as long. The statistical evidence for the reduction is overwhelming and robust to every check we planned; its size is limited not by chance but by the verifier's coverage (Petstore bodies, task fidelity, curated specifications) and by the model's ability to correct itself.

10 Threats to Validity

We follow the standard classification of threats in experimental software engineering (Wohlin et al., 2012): construct, internal, external and conclusion validity, followed by reproducibility and practical trade-offs. Table 25 summarises the most important threats and what we did about each.

10.1 Construct validity: are we measuring the right thing?

- **Hallucination means contract violation.** Class A measures conformance to a frozen specification, not whether a live service would accept the request. Some APIs silently ignore unknown parameters (DummyJSON ignores `sort`), so a class A candidate may "work" in the sense of returning data; it still does not do what the code claims, which is the harm we target.
- **Withholding is not hallucinating, but it is not answering either.** An agent that refused everything would score zero hallucinations. We guard against this by treating correct code delivered as a co-primary concern (H2), by reporting abstention and incorrect rejection, and by showing that the agent delivered *more* correct code than the baseline.
- **Ground truth is partly researcher-authored.** Three of the six specifications are curated partial contracts, and the planned two-reviewer audit of a 100-task sample of the answer key was not completed. The evaluator's self-check (1,000/1,000 reference solutions correct) and the documented review of every case the curated specifications left open reduce, but do not remove, this threat.
- **The EWCS is evidence coverage, not probability.** We never interpret it as a probability, and Section 8.6 shows that doing so would be wrong.
- **Behaviour is checked narrowly.** Three behaviour checks (value, empty response, error) do not establish that a program is fully correct; both conditions perform poorly on empty-response handling.

10.2 Internal validity: is the intervention the cause?

- **Context in the prompt.** The Confident Agent prompt names the hosts with frozen evidence, so part of the drop in wrong-host hallucinations is due to context, not verification. We designed the intervention this way deliberately (it is part of the package a developer would run) and quantified it: hallucinations other than a wrong host fell by 74–79% (Section 8.4).
- **Independent samples per condition.** Each condition generated its own code, so baseline and intervention answers differ by sampling as well as by treatment. Pairing by task, large effect sizes relative to the ±2-point run-to-run variation (Section 8.11), and three replications limit this threat.
- **Model-controlled protocol.** The model deviated from the prompt in 46–47% of tasks. Because the tool's decision on the last verified candidate determines what reaches the user, and no unapproved code was ever shown, deviations affect cost but not the primary outcome.
- **Evaluator bias.** The same team, with AI assistance, built the agent and the evaluator. We separated them by code (no shared modules), by method (sandboxed execution versus static analysis) and by data (only the evaluator reads the answer key), and validated the evaluator on the reference solutions. The planned stratified hand audit of 50 decided candidates per mode was not completed.

- **A verifier change after run 1.** One verifier rule (undeclared body fields) was changed after inspecting run 1. It is disclosed in the evaluation plan, it applies to every reported run, and it made the verifier stricter, not more favourable to the hypothesis.
- **The manual review was AI-assisted.** The first review of 396 undetermined candidates used AI assistance; an independent human second review of the 46 decisive cases agreed on 43 (κ = 0.88), and every decision is recorded with its evidence so that any reader can audit it. Three cases were relaxed from B to correct by group decision.
- **Environment drift.** The first series of runs was disrupted by a sustained throughput drop on 29 September, and LM Studio's GPU runtime updated itself automatically twice after that series. The runtime version is not recorded in the run manifests, so we cannot prove that the three final runs, made on consecutive days, used an identical runtime. Accuracy was unaffected by the earlier fault (Section 8.11).

10.3 External validity: how far do the results generalise?

- **One model.** Qwen3.5-9B at three quantizations is one model family at one size. Larger models hallucinate less and follow tool protocols better; a different model could shift both the baseline rate and the cost.
- **Six popular public APIs.** Two are fake-data services (JSONPlaceholder, DummyJSON) common in tutorials, which the model knows unusually well, and one (REST Countries v5) is unusually new. Large enterprise APIs, private APIs and APIs without any specification may behave differently.
- **Python with `requests`, generated prompts, single requests.** Other languages, HTTP clients, sessions, asynchronous code and multi-step workflows are outside the toolbox's scope and abstain.
- **Only answerable tasks.** The benchmark has no ambiguous or impossible requests, so it cannot measure whether the agent correctly refuses a task that should be refused; it measures only whether it withholds code it cannot verify.
- **Specifications are a precondition.** The approach requires a frozen specification for every API it verifies. Four of our six APIs publish no adequate official one, and for an unknown host the agent always abstains.
- **One machine, one tariff.** Times and energy are specific to an AMD RX 9070 XT and Ryzen 7 5700X3D, and the Rand values to the Durban Central tariff of September 2026; the measured energy excludes wall-socket overheads.

10.4 Conclusion validity: are the statistics sound?

Each task was sampled once per condition and run, at temperature 0.2 without a fixed seed, so individual task outcomes are noisy (11–14% of tasks changed status between repeated runs). We therefore draw conclusions only from paired aggregate tests, use exact rather than asymptotic McNemar tests, report cluster-bootstrap intervals over operation families to allow for related prompts, and apply Holm's correction across runs. Subgroup results and quantization differences are presented descriptively. The primary effect is so large relative to its uncertainty that no plausible correction changes the conclusion; smaller differences, such as between quantizations, should not be over-interpreted.

10.5 Reproducibility

All inputs are frozen and identified by hash: the task and prompt files, every provider specification, the agent's system prompt and the frozen package evidence. Every run is stored in full, every candidate byte for byte, and every statistic, table and chart in this report is regenerated from those files by scripts in the submission (Appendix G). What cannot be reproduced exactly is the model's sampling (no seed was set) and, possibly, LM Studio's runtime version; a rerun would reproduce the aggregate results within run-to-run variation rather than task by task.

10.6 Practical trade-offs

The intervention is not free. It multiplies latency by about six and processed tokens by 40–46, withholds code for 38–40% of requests, and on one API (REST Countries) withholds everything. Electricity is cheap at local scale (about one cent per verified prompt), but on a pay-per-token cloud model the 40-fold token increase would dominate the cost. Whether the trade is worth it depends on the harm of a hallucinated integration reaching production, which in most settings is far larger than a minute of waiting.

Table 25**:** Summary of the main threats to validity and their mitigation.

Threat	Risk	Mitigation
Prompt names current hosts	Improvement attributed to verification is partly context	Quantified with non-host hallucinations (−74% to −79%)
Evaluator built by the same team	Correlated blind spots	Separate code and method; sandboxed execution; self-check on 1,000 references
Curated partial specifications	Ground truth errors	Labelled; undetermined cases reviewed against documentation and live APIs; κ = 0.88
Answer-key review incomplete	Systematic answer-key errors undetected	Self-check; review of open cases; disclosed as a limitation
One sample per task	Noisy task outcomes	Paired aggregate tests; three replications; repeat-run agreement reported
Related prompts	Overstated precision	Cluster bootstrap over 141 operation families
Weights and thresholds chosen by us	Result depends on arbitrary numbers	Five pre-planned schemes: no change to withheld or delivered outcomes
Runtime drift	Unequal conditions between runs	Final series on consecutive days; first series reproduces accuracy within ~2 points
One model, six APIs, Python	Limited generality	Stated scope; design is API-agnostic but evidence-dependent

11 Conclusion

We asked whether an agent that verifies the REST API claims in its own code against frozen, authoritative specifications can stop API hallucinations from reaching the user, and at what cost. On a frozen benchmark of 1,000 Python tasks, judged by an independent evaluator and replicated at three quantizations, the answer is yes. Across 2,970 paired tasks, hallucinated code reached the user in 52.6% of tasks without the agent and in 2.0% with it (exact McNemar $p < 10^{-129}$ in every run), while correct code reaching the user rose from 32.1% to 50.9%. The scientific hypothesis of the brief is supported, and the result survives every robustness check we planned.

Four findings go beyond the headline numbers.

1. **The model writes the API it was trained on.** Its hallucinations are dominated by retired and extrapolated versions and by confusion between similar services. These errors are invisible to the model and obvious to a pinned specification, which is why external evidence, not self-reflection, is the right tool.
2. **Context helps, verification decides.** Naming the current hosts in the prompt removed most wrong-host errors, but the model still invented the rest of the URL; hallucinations other than a wrong host fell by three-quarters only because the verifier contradicted them.
3. **Safety comes from hard rules, not from the score.** Under every planned weighting scheme the same code was withheld and the same code delivered; removing the uncertainty gate would have let almost four times as many hallucinations through. The EWCS discriminates well (AUC 0.93–0.94) but is a measure of evidence coverage, not a probability, and the design does not rely on it for safety.
4. **Trust has a price.** The agent abstains on about four tasks in ten, takes about six times as long, and processes 40–46 times as many tokens. At local electricity prices that is roughly one cent per prompt; the real costs are latency and GPU time.

The design also has clear limits. Its residual hallucinations come from values that are only known at run time, it does not check whether a valid call is the one the task wanted, the model repairs its own code only about a third of the time, and it can verify only APIs for which a specification has been frozen. These limits point directly to future work: binding representative inputs during verification; adding task-level checks for the intended operation; trimming the conversation and capping correction rounds to cut cost; completing the human review of the answer key; repeating the experiment with seeded samples and other models; adding ambiguous and impossible tasks so that correct refusal can be measured; and monitoring frozen specifications for drift against their live versions.

The brief's guiding sentence was that a Confident Agent is not one that is always correct, but one that knows when it should not be trusted. Our agent is wrong far less often than the model it wraps, and when it cannot establish that its code is right, it says so instead of handing it over. That behaviour does not come from the model becoming more humble: it is a property of the architecture, trust by design rather than by declaration (Vorster, 2026b).

AI Disclosure

In line with the project description's academic-integrity requirements (University of KwaZulu-Natal, 2026, §A.4), we acknowledge the following use of generative AI.

- **Design discussions and early drafts.** DeepSeek and ChatGPT were used to explore the API domain, draft early dataset ideas and discuss verifier designs, including the trade-off between dynamic specification discovery and frozen evidence. The conversation transcripts are retained.
- **Software.** Claude (Anthropic), used through Claude Code, assisted in writing, testing and debugging the dataset builder, the Confident Agent toolbox, the benchmark harness, the evaluator, the statistics compiler and the analysis and diagram scripts.
- **Review.** The first manual review of the 396 undetermined candidates was AI-assisted (Claude), with every rule and decision recorded; a group member independently reviewed the 46 decisive candidates (Section 7.5).
- **This report.** The report was drafted with AI assistance (Claude) from the group's code, data, evaluation plan, earlier reports and the lecturer's feedback. Every number, table and chart is generated by scripts in the submission from the recorded experimental data.

All research decisions, including the domain, the hypotheses, the evaluation plan and its changes, were discussed and taken by the group, and the group takes responsibility for the content of this report.

References

Barron, F.H. and Barrett, B.E. (1996) 'Decision quality using ranked attribute weights', *Management Science*, 42(11), pp. 1515–1523.

Cheng, J., Marone, M., Weller, O., Lawrie, D., Khashabi, D. and Van Durme, B. (2024) 'Dated data: Tracing knowledge cutoffs in large language models', *Conference on Language Modeling (COLM)*. arXiv:2403.12958.

Cohen, J. (1960) 'A coefficient of agreement for nominal scales', *Educational and Psychological Measurement*, 20(1), pp. 37–46.

DummyJSON (2026) *DummyJSON documentation: products, users, posts, todos*. Available at: https://dummyjson.com/docs (Accessed: 21 September 2026).

Efron, B. and Tibshirani, R.J. (1993) *An Introduction to the Bootstrap*. New York: Chapman & Hall.

eThekwini Municipality (2026) *Electricity account for the test site, Durban Central, September 2026: energy charge R3.5747 per kWh excluding VAT*. Durban: eThekwini Municipality.

Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis. University of California, Irvine.

GitHub (2026) *GitHub REST API description (api.github.com OpenAPI document)*. Available at: https://github.com/github/rest-api-description (Accessed: 21 September 2026).

Holm, S. (1979) 'A simple sequentially rejective multiple test procedure', *Scandinavian Journal of Statistics*, 6(2), pp. 65–70.

Huang, L. et al. (2025) 'A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions', *ACM Transactions on Information Systems*, 43(2). arXiv:2311.05232.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A. and Fung, P. (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), Article 248.

JSONPlaceholder (2026) *JSONPlaceholder guide*. Available at: https://jsonplaceholder.typicode.com/guide/ (Accessed: 21 September 2026).

Landis, J.R. and Koch, G.G. (1977) 'The measurement of observer agreement for categorical data', *Biometrics*, 33(1), pp. 159–174.

LibreHardwareMonitor (2026) *LibreHardwareMonitor: open-source hardware monitoring*. Available at: https://github.com/LibreHardwareMonitor/LibreHardwareMonitor (Accessed: 27 September 2026).

Liu, F. et al. (2024) 'Exploring and evaluating hallucinations in LLM-powered code generation'. arXiv:2404.00971.

LM Studio (2026) *LM Studio: local large language model server*. Available at: https://lmstudio.ai (Accessed: 30 September 2026).

McNemar, Q. (1947) 'Note on the sampling error of the difference between correlated proportions or percentages', *Psychometrika*, 12(2), pp. 153–157.

Microsoft (2026) *Pyright: static type checker for Python*, version 1.1.414. Available at: https://github.com/microsoft/pyright (Accessed: 25 September 2026).

Open-Meteo (2026) *Open-Meteo OpenAPI specifications (forecast, air quality, marine)*. Available at: https://github.com/open-meteo/open-meteo/tree/main/openapi (Accessed: 21 September 2026).

OpenAPI Initiative (2021) *OpenAPI Specification, version 3.1.0*. Available at: https://spec.openapis.org/oas/v3.1.0 (Accessed: 21 September 2026).

Patil, S.G., Zhang, T., Wang, X. and Gonzalez, J.E. (2023) 'Gorilla: Large language model connected with massive APIs'. arXiv:2305.15334.

Qwen Team (2026) *Qwen3.5-9B* [GGUF model weights, quantizations Q4_K_M, Q6_K and Q8_0]. Distributed through LM Studio (Accessed: 30 September 2026).

REST Countries (2026) *Countries API v5 documentation*. Available at: https://restcountries.com/docs/countries (Accessed: 21 September 2026).

Spracklen, J. et al. (2025) 'We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs', *34th USENIX Security Symposium*. arXiv:2406.10279.

Stillwell, W.G., Seaver, D.A. and Edwards, W. (1981) 'A comparison of weight approximation techniques in multiattribute utility decision making', *Organizational Behavior and Human Performance*, 28(1), pp. 62–77.

Swagger (2026) *Swagger Petstore: OpenAPI 3 sample*. Available at: https://petstore3.swagger.io/api/v3/openapi.json (Accessed: 21 September 2026).

The Skeptics (2026a) *COMP301 Confident Agent: Progress Report One*. Unpublished group report, University of KwaZulu-Natal.

The Skeptics (2026b) *COMP301 Confident Agent: Progress Report Two, static verification of API hallucinations in LLM-generated code*. Unpublished group report, University of KwaZulu-Natal.

University of KwaZulu-Natal (2026) *COMP301 (2026) Software Design. Confident Agent: Designing for Trust under Uncertainty*. Group project assignment, version 1.0. School of Mathematics, Statistics and Computer Science.

Vorster, L. (2026a) *Trust by Design: Architectural Patterns for Confident AI Agents*. COMP301 course notes, University of KwaZulu-Natal.

Vorster, L. (2026b) *Trust by Design, not by Declaration*. COMP301 executive presentation, University of KwaZulu-Natal, 1 October.

Vorster, L. (2026c) Feedback on Progress Report Two to The Skeptics [email]. Personal communication.

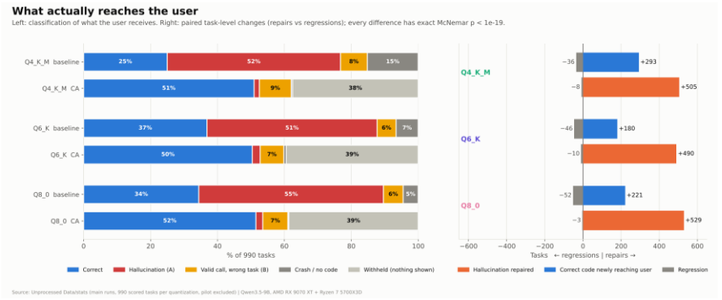
Wilcoxon, F. (1945) 'Individual comparisons by ranking methods', *Biometrics Bulletin*, 1(6), pp. 80–83.

Wilson, E.B. (1927) 'Probable inference, the law of succession, and statistical inference', *Journal of the American Statistical Association*, 22(158), pp. 209–212.

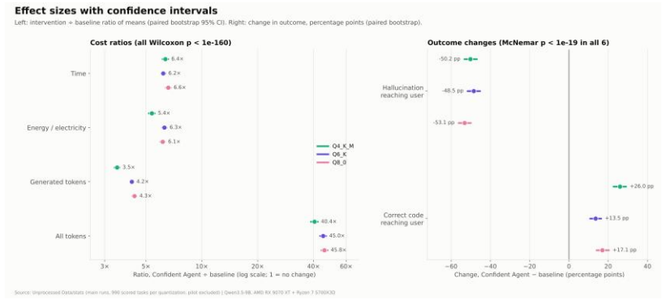
Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B. and Wesslén, A. (2012) *Experimentation in Software Engineering*. Berlin: Springer.

Appendix A: Summary of Results

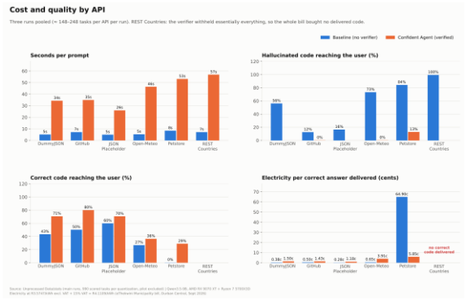
Supplementary charts generated from the unprocessed data (`Findings/Statistical Analysis`). All panels use the 990 scored tasks per quantization (pilot excluded) and price electricity at R4.1109/kWh including VAT. Baseline is blue and the Confident Agent orange throughout.



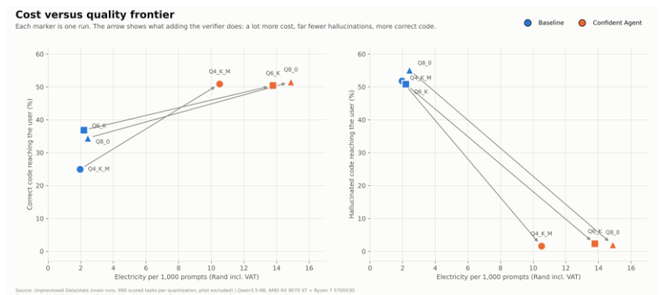
(a) What reaches the user, with paired repairs and regressions



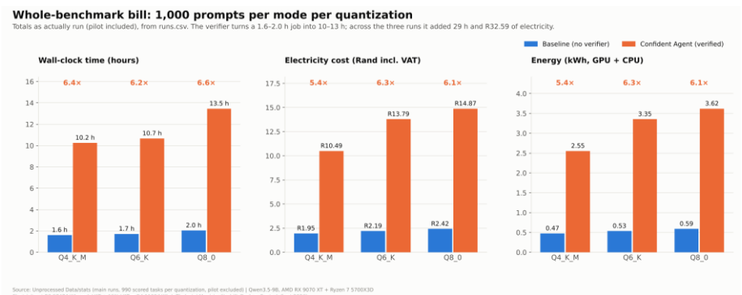
(b) Effect sizes: cost ratios and outcome changes



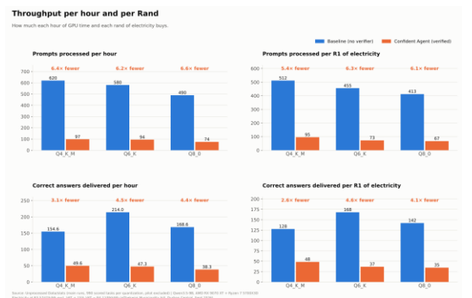
(c) Cost and quality by API



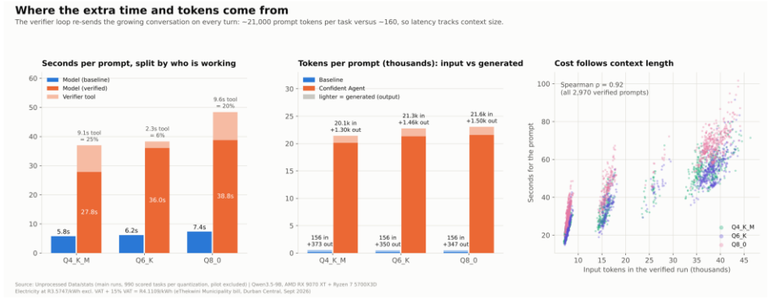
(d) Cost versus quality frontier



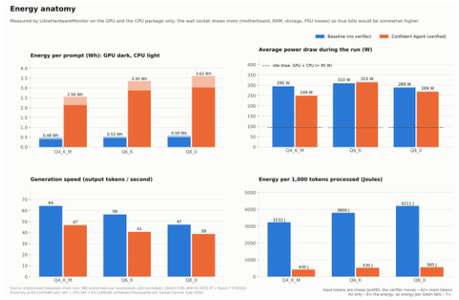
(e) Whole-benchmark time and electricity bill



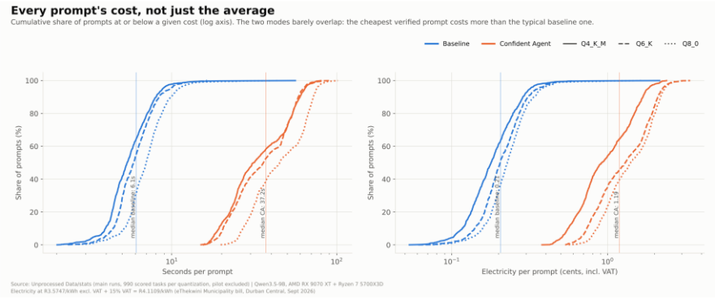
(f) Throughput per hour and per Rand



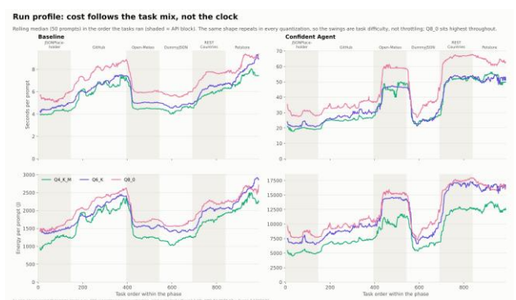
(g) Where the extra time and tokens come from



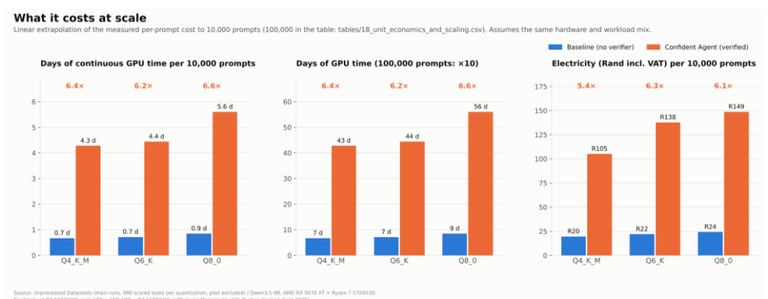
(h) Energy and power anatomy



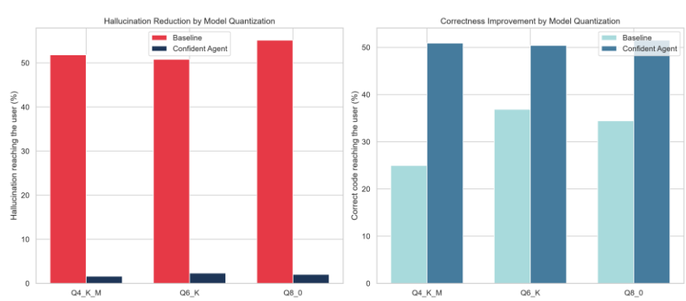
(i) Distribution of per-prompt cost



(j) Run profile: cost follows the task mix



(k) Linear extrapolation to 10,000 and 100,000 prompts



(l) Hallucination and correctness by quantization

Figure 23: Summary of results: supplementary charts (a)–(l).

Appendix B: Changes since Progress Report Two

Progress Report Two was written before implementation. This report supersedes it on the following points; the reason is given for each.

Table 26**:** Design decisions in Progress Report Two and in the final system.

Topic	Progress Report Two	Final	Why
Model	Qwen3.5-9B-Instruct; temperature 0.3; top-p 0.9; context 8,192; fixed seed	Qwen3.5-9B GGUF at Q4_K_M, Q6_K, Q8_0; temperature 0.2; context 30,208; no seed; thinking off	Tool conversations need a longer context; one pinned configuration for every run (lecturer item 2)
Baseline prompt	"You are a code generation assistant. Provide only code."	Two-line API coding-assistant prompt asking for one python block	Reliable code-block extraction
Evidence	Specifications "available"; later dynamic discovery considered	Frozen, hashed snapshots bundled with the toolbox and selected by host	Reproducibility; four APIs publish no discoverable specification (lecturer item 4)
Static checker	Pylance via the Language Server Protocol	Pyright 1.1.414 command line (Pylance's engine)	Scriptable and pinnable
Tool attachment	MCP tool	LM Studio tools provider; function calling through a bridge in the experiment	Same interface without an extra server (Section 6.3)
Layer 2 tier	typeshed via Pylance, tier 2 (0.8) in the worked example	Absence of a Pyright error, tier 4 (0.5)	A clean type check is weak evidence
Missing or failed layer	Excluded from the score (Python reference)	Counted as unverifiable; the decision abstains	A layer that did not run must not look like a clean result
Decision policy	Safety constraint, then thresholds 0.85/0.60	Safety constraint, **uncertainty gate**, then thresholds	Without the gate, unverifiable candidates were delivered (Section 8.7)
Correction	At most one, by the controller	At most one, by the model under the prompt; deviations measured	Deployable package; tool keeps the delivery decision
Primary metric	Unsafe Delivery Rate	Class A code reaching the user, per task, by an independent evaluator	Same idea, defined with explicit classes A, B, C
Evaluation	Static comparison and H01–H10 codes	Sandboxed execution with a recording stand-in; H-codes mapped to error kinds	Exact requests, including conditional code (lecturer item 5)
Dataset targets	40/40/20% difficulty; 55% no authentication; 10% manual review	Actual composition reported; 100-task review not completed	Do not relabel tasks to hit targets
Statistics	McNemar (pending research)	Exact McNemar, task and family bootstrap, Holm, Wilson, Wilcoxon, AUC, κ	Paired binary outcome; related prompts
Sensitivity analysis	Planned (five schemes)	Done, plus the gate (Section 8.7)	Lecturer item 6
Multi-agent reframing	Chief agent with specialist agents	One model, one deterministic verification tool	Deterministic checks are stronger evidence than another model

Appendix C: Example Task and Prompts

Task `REST_PY_GH_0003` (operation family `06ffd07d…`, difficulty heuristic "medium", provider specification `github.openapi.json`, SHA-256 `4fbdc7d0…`). The model sees only the instruction:

```
Write a Python function solve(inputs) for GitHub REST API version 2026-03-10. Delete an
organization. The inputs dictionary provides: organization: The organization name. Use the
caller-supplied credential in inputs["credential"], assumed to have the necessary
permissions.
Return the decoded JSON response. Return None for a successful empty response. Raise an
exception
for an unsuccessful HTTP response. Assume referenced resources and accounts exist. Do not
execute
the function.
```

The answer key holds the expected request for the test inputs `{"organization": "example-org", "credential": "EXAMPLE_CREDENTIAL_NOT_REAL"}`:

```
DELETE https://api.github.com/orgs/example-org
Accept: application/vnd.github+json
X-GitHub-Api-Version: 2026-03-10
Authorization: Bearer EXAMPLE_CREDENTIAL_NOT_REAL
```

and a reference solution, generated from the specification by template:

```
import requests
from urllib.parse import quote

def solve(inputs):
    url = 'https://api.github.com/orgs/{org}'
    headers = {}
    headers['Accept'] = 'application/vnd.github+json'
    headers['X-GitHub-Api-Version'] = '2026-03-10'
    headers['Authorization'] = 'Bearer ' + inputs["credential"]
    url = url.replace('{org}', quote(str(inputs['organization']), safe=''))
    response = requests.request('DELETE', url, params={}, headers=headers, timeout=20)
    response.raise_for_status()
    if response.status_code == 204 or not response.content:
        return None
    return response.json()
```

The Confident Agent system prompt (194 lines, in `Code Submission/confident-agent-toolbox/instructions`) prescribes this workflow: write the complete candidate with `requests`; do not look for or write a specification (evidence is selected by host from the eight listed hosts); call `verify_candidate` exactly once with the code only; if a layer is `verified_incorrect` on the first candidate, fix exactly those problems and verify once more; otherwise, or after the second result, reply with the tool's `report` verbatim. It states that the tool does not check task behaviour and that the EWCS is not a probability.

Appendix D: Run Artefacts and Provenance

Table 27**:** Identifiers of the frozen inputs (SHA-256 prefixes; full hashes in the run manifests and the dataset).

Artefact	SHA-256	Notes
Task file (answer key) rest_api_tasks_v1_DRAFT.jsonl	7a09f2bb…	LF line endings
Prompt file rest_api_prompts_v1_DRAFT.jsonl	c4c68a64…	identical in both conditions
Confident Agent system prompt (as sent)	0ebc4dce…	operator note removed
Frozen PyPI evidence (requests)	ef06670a…	interface layer
github.openapi.json	4fbdc7d0…	13.0 MB, 819 paths
petstore.openapi.json	00441c05…	OpenAPI 3.0.4
openmeteo-forecast / air-quality / marine	f57c2918… / ee203da7… / 096a28c7…	converted from YAML
jsonplaceholder / dummyjson / restcountries (curated)	d2dfbc53… / d313813a… / d02f4ab3…	researcher-authored subsets

Run folders (`Findings/Benchmark/Completed Runs/<model>/`) hold one zip per condition with every reply, tool call, verified candidate and verification result, and a run manifest recording the settings, LM Studio's model information, the machine and the power sensors. The final runs were started from repository commits `d954dc0` (Q6_K), `f6ee6b7` (Q8_0) and `7e928f2` (Q4_K_M); the toolbox and benchmark code are identical in all three (benchmark 2026-09-27.2, toolbox 3.0.0).

Appendix E: Error Kinds Used by the Evaluator

Table 28**:** Kinds of error recorded in the code reaching the user, read from the evaluator's findings with fixed rules.

Class	Kinds
A	wrong host, version or base path; invented endpoint or method; invented parameter or body field; invalid value or structure; missing required part or credential; undocumented response field; non-existent `requests` API
B	different declared endpoint; method not acceptable for the task; omitted parameter the task specifies; different parameter value; omitted request body; unrequested request body; request body differs; extra requests; credential missing or malformed where the API's requirement is unknown; JSON body sent as form data; insecure scheme (http)
Runtime failure	input used as the wrong type; reads an input the task does not provide; syntax error; makes no request; does not finish (timeout); blocked operation; undefined name; raises its own error before the request; wrong call or argument types

Appendix F: Manual Review Rules

Each question that a curated specification left open was answered once and applied to every candidate asking it (`Findings/EvaluateBenchmark/reviews/review_rules.py`). The rules, with their evidence:

- **DummyJSON query parameters.** The documentation lists `limit`, `skip`, `select`, `sortBy`, `order` and `q`. Names such as `sort`, `sort_by`, `page`, `fields`, `include` and `_fields` are class A; a live check showed they have no effect (`?sort=title` returns the unsorted list, `?sortBy=title` sorts it).
- **DummyJSON and JSONPlaceholder body fields.** A field the resource has (for example `gender` on a user, `id` on a todo) is an extra documented body field, class B; a field it does not have (for example `title` on a DummyJSON todo, `userId` on a JSONPlaceholder photo) is class A.
- **Response fields.** DummyJSON returns the resource object itself; reading `users`, `post`, `success`, `statusCode` or `content` from a single-resource response is class A.
- **JSONPlaceholder filters and routes.** Filtering by any resource field is documented ("basic filtering is supported through query parameters") and works live; filtering by a non-field (`ids`) is ignored and class A. Nested routes such as `/posts/1/comments` are documented and real; using one where the task expected another endpoint is class B.
- **REST Countries paths.** Every v5 endpoint lies under `/countries/v5` (list, search, aggregate, read and search by property); paths such as `/v5/countries`, `/v5.0/search` or `/v3.1/all` on `api.restcountries.com` do not exist and are class A.

The first review classified the 46 candidates that reached the user as 24 class A and 22 class B; the second reviewer agreed on 43, and the group settled the three JSONPlaceholder list-endpoint cases as correct. Of the 350 withheld candidates, 327 were class A and 23 class B.

Appendix G: Reproducing the Results

The submission folder `COMP301 Project Final Submission` contains `Code Submission` (toolbox, dataset builder), `Findings` (benchmark with completed runs, evaluator, statistical analysis) and `Final Report and Documentation`. The evaluator and statistics need only Python 3.9 or later; the charts need pandas, NumPy, SciPy, Matplotlib and seaborn.

```
cd "COMP301 Project Final Submission\Findings\EvaluateBenchmark"
python evaluate_benchmark.py              # sandboxed scoring of every run (about 12
minutes)
python compile_stats.py                   # CSV tables in Statistical Analysis\Unprocessed
Data
cd "..\Statistical Analysis"
python -X utf8 tempstats\build_tempstats.py                # cost charts and tables
python -X utf8 tempstats\report_figures\build_report_figures.py   # charts R01-R05
cd "..\..\Final Report and Documentation\Final Report\build"
python -X utf8 build_diagrams.py          # diagrams D01-D08
node build_report.js report.docx          # this report (then update fields in Word)
```

Rerunning the benchmark itself requires LM Studio with the Qwen3.5-9B quantizations, the toolbox's Python environment and LibreHardwareMonitor for energy; the procedure is in `Findings/Benchmark/README.md`.

Appendix H: Glossary

Table 29**:** Glossary.

Term	Meaning
Abstention	The agent withholds code because evidence contradicts it or cannot establish it.
Baseline	The same model with a minimal system prompt and no verification.
Class A / B / C	API hallucination / valid call that is wrong for the task / equivalent to the expected request (Table 1).
Confident Agent	The model plus the Confident Agent prompt and the `verify_candidate` tool.
EWCS	Evidence-Weighted Confidence Score: the share of applicable verification weight supported by positive evidence. Not a probability.
Frozen evidence	Specification files stored with their hash and never changed during the experiment.
Incorrect acceptance / rejection	A hallucinated candidate delivered / a correct candidate withheld.
McNemar test	Test for a change in a paired binary outcome, using only the discordant pairs.
Operation family	A group of tasks that exercise the same underlying API operation.
Uncertainty gate	The policy rule that withholds code when any applicable verification layer is unverifiable.
Verified correct / incorrect / unverifiable / not applicable	The four states of a verification layer.